

Automated Certificate Enrollment with EST

Deploying OpenXPKI and Cisco IOS XE for RFC 7030 Enrollment

XPKI-EST-XE Project

April 22, 2026

ABSTRACT

This deployment guide walks through building a working Enrollment over Secure Transport (EST, RFC 7030) environment using the OpenXPKI Community Edition certificate authority on Ubuntu Linux and Cisco IOS XE routers as the EST client.

The guide was built end-to-end in a reference lab. Every command, policy file, and router configuration was verified against a working deployment consisting of OpenXPKI 3.x running in Docker on Ubuntu 24.04 LTS and a Cisco Catalyst 8000V running IOS XE 17.18.2.

Table of Contents

Introduction	4
What you get from this guide	5
How the device authenticates across its lifetime	5
Scope	6
How this guide is organized	6
Quick start.....	7
Reference topology.....	8
Versions tested	8
Security posture of the lab	8

Chapter 1. Prerequisites	10
Hosts	10
Networking	10
DNS	11
NTP.....	11
Operating-system packages on the CA host	11
SSH access model used in this guide.....	11
Firewall and security considerations	12
Hardening notes (for later)	12
Chapter 2. OpenXPki Installation on Ubuntu	13
1. Install Docker Engine	13
2. Clone the openxpki-docker repository	13
3. Prepare the datavault secret	14
4. Configure the oxi CLI authentication	14
5. Start the stack.....	15
6. Smoke test the web front-end.....	15
7. Useful operational commands.....	15
8. What the web container mounts.....	16
Chapter 3. Bootstrapping the PKI	17
1. Decide on labels and validity	17
2. Run the sample-config script	17
3. Verify the realm	18
4. Confirm certificates in the database.....	18
5. Check the web UI	18
6. Export the Root CA for IOS XE.....	18
7. (Optional) Trust the Root on your workstation	19
8. Set the CRL Distribution Point (CDP) URL	19
9. What we did not do	20
Chapter 4. TLS Server Certificate for the EST Endpoint	21
1. Generate a key and CSR on the CA host	21
2. Submit the CSR via EST (eat your own dog food).....	21
3. Install the certificate into Apache.....	23

4. Validate the TLS endpoint.....	23
5. What tls_server vs tls_dual gives you.....	24
6. Rotation	24
Chapter 5. EST Endpoint Configuration in OpenXPKI.....	25
1. Endpoint policy (est/default.yaml)	25
2. The tls_dual certificate profile.....	27
3. Apache in front of the EST endpoint.....	30
4. Sanity checks.....	34
5. What is still worth reviewing for production	35
Chapter 6. IOS XE EST Client Configuration	36
1. Prerequisites on the router.....	36
2. Generate a dedicated RSA key pair.....	36
3. Authenticate the Root CA	37
4. Configure the EST enrollment profile	38
5. Configure the device (client) trustpoint.....	40
6. Authenticate the EST server (pull CA chain via EST)	42
7. Enroll for the device certificate.....	42
8. Re-enrollment and renewal	43
9. Using the issued certificate.....	43
Chapter 7. Testing and Verification	44
1. OpenXPKI stack health	44
2. EST endpoints via curl.....	44
3. TLS server verification (hostname + chain)	45
4. IOS XE state checks	45
5. End-to-end transaction trace.....	46
6. Revocation check	47
7. Re-enrollment testing.....	47
8. Clean-up after a test burst	50
Chapter 8. Troubleshooting and Common Issues.....	51
Stack start fails.....	51
curl to /cacerts returns 403 or 500	51
simpleenroll returns 401 Unauthorized.....	52

EST request returns 202 Pending.....	52
oxi workflow commands fail with ACCESS_NOT_ALLOWED_FOR_USER.....	53
Issued cert missing SAN or Client ECU	53
crypto pki authenticate EST-CLIENT fails	53
Enroll keeps failing with SIGNER_NOT_AUTHORIZED.....	54
IOS XE “Include an IP address” prompt freezes automation	55
Clock skew errors.....	55
Hostname mismatch during TLS handshake.....	55
Reenrollment fails with TLS handshake failure.....	56
Reenrollment returns the same certificate	56
Reenrollment rejected as outside the renewal window.....	57
Auto-regenerate fails with “Querying key pair failed”	57
Cant change RSA key type here	58
OpenXPki refuses to parse the CSR	58
Useful log excerpts to grep	58
Quickly reset the lab	59
Chapter 9. Day-Two Operations	60
Reload vs restart	60
Log locations	60
Backup	60
Certificate housekeeping	61
Renewal on the router	61
Updating OpenXPki	62
Credential rotation	62
Hardening review checklist.....	63

Introduction

A comprehensive deployment guide and reference project for setting up [Enrollment over Secure Transport \(RFC 7030\)](#) between an [OpenXPki](#) Certificate Authority running on Ubuntu Linux and a Cisco IOS XE device acting as the EST client.

The guide was built from an end-to-end lab where a Cisco Catalyst 8000V running IOS XE 17.18.2 successfully obtained a device identity certificate from an OpenXPKI 3.x instance deployed via Docker Compose on Ubuntu 24.04 LTS. The goal of this guide is to provide a repeatable recipe and a set of reference configurations so that the same result can be reproduced in your own lab or production environment.

What you get from this guide

- A working OpenXPKI Community Edition CA exposing RFC 7030 EST endpoints over TLS on port 8443 (`/well-known/est/cacerts`, `/simpleenroll`, `/simplereenroll`).
- A two-tier PKI hierarchy (Root CA and Issuing CA) bootstrapped from the OpenXPKI sample scripts.
- A CA-signed TLS server certificate on the EST endpoint that IOS XE will trust and whose hostname verification succeeds.
- Clear OpenXPKI `tls_dual` profile and endpoint policy configuration that produces correctly-formed device certificates (Subject, SAN, Server/Client ECU).
- A Cisco IOS XE configuration that automatically pulls the CA chain via `crypto pki authenticate` and enrolls for a device certificate via `crypto pki enroll`.
- A troubleshooting section that captures the exact failure modes we hit during the lab build and how to resolve them.

How the device authenticates across its lifetime

RFC 7030 defines three EST operations, and this guide uses a different authentication method for each one. Understanding the split up front makes the rest of the guide much easier to follow, because every chapter - Apache config, OpenXPKI policy, IOS XE profile - is just one half of the following picture:

EST operation	When the router calls it	Router authenticates with	Server enforces
GET <code>/cacerts</code>	Before every enroll / renew, to refresh the CA chain	Nothing (anonymous)	Nothing
POST <code>/simpleenroll</code>	Once , the very first time the device enrolls	HTTP Basic (enrollment <code>http username ... password ...</code>)	Apache <code>AuthType Basic</code> + <code>htpasswd</code>
POST <code>/simplereenroll</code>	Every renewal thereafter	mTLS with the device's current certificate	Apache <code>+OptRenegotiate + SSLVerifyClient</code>

The lifecycle view is:

1. **Day 0 - first boot.** The router has no certificate and nothing mTLS-worthy to present. It POSTs to `/simpleenroll` carrying its `Authorization: Basic` header. Apache validates the credential against `htpasswd/est.passwd` and, if valid, proxies the CSR to OpenXPKI. The backend auto-approves and returns the first certificate.

2. **Day ~38 (80 % of a 47-day cert) - first renewal.** IOS XE fires the `auto-enroll 80` regenerate timer. The router generates a fresh key, builds a new CSR, opens a TLS 1.2 session to the CA, presents its *current* device certificate via mTLS, and POSTs to `/simpleenroll`. No Basic header is transmitted - IOS XE omits it on `/simpleenroll` by design (verified on the lab). OpenXPki issues a new cert with a new serial number and a new public key.
3. **Every subsequent renewal.** Same as step 2, forever. The Basic credential is never sent again unless the device has to bootstrap from scratch (zeroize, config wipe, RMA replacement, or a cert that expired before the renewal window closed).

Rotating the Basic credential on the CA therefore affects only devices that still have to bootstrap; it does not interrupt the in-flight population. See Chapter 6 section 4a for the empirical evidence and the recommended two-hash rollover pattern.

Scope

This is a lab deployment guide intended to get you from “nothing” to “working EST enrollment” in a controlled environment. The configuration that the guide produces is already tightened along the axes that matter for everyday production use (per-device HTTP Basic bootstrap credentials, mTLS on `/simpleenroll`, an FQDN allow-list on the certificate profile, a pinned `authorized_signer` rule, CRL publication, and a full set of security headers on the WebUI). The remaining lab-specific choices - the DUMMY CA hierarchy seeded by OpenXPki’s `sampleconfig.sh`, the software `svault` token, and running the CRL fetch endpoint over plain HTTP - are called out inline where they appear so you know exactly what to swap for a production cutover.

This guide does not cover: - High availability or clustering of OpenXPki. - Integration with external HSMs or smartcard-based RA operators. - TLS 1.3 post-handshake client authentication, which is not currently used by the IOS XE EST client.

How this guide is organized

The guide is divided into nine chapters. Each chapter covers one phase of the build and is self-contained enough to be revisited on its own when troubleshooting or refreshing a step later.

Chapter	Content
Chapter 1 - Prerequisites	Host requirements, network layout, DNS, NTP, and required packages.
Chapter 2 - OpenXPki Installation on Ubuntu	Install Docker and deploy OpenXPki via the <code>openxpki-docker</code> compose project.
Chapter 3 - Bootstrapping the PKI	Generate the Root and Issuing CAs, configure the <code>datavault</code> , and bootstrap the <code>democa</code> realm.
Chapter 4 - TLS Server Certificate for the EST Endpoint	Issue the CA-signed TLS certificate for the EST endpoint and wire it into Apache.
Chapter 5 - EST Endpoint Configuration in OpenXPki	Configure the EST endpoint policy, certificate profile,

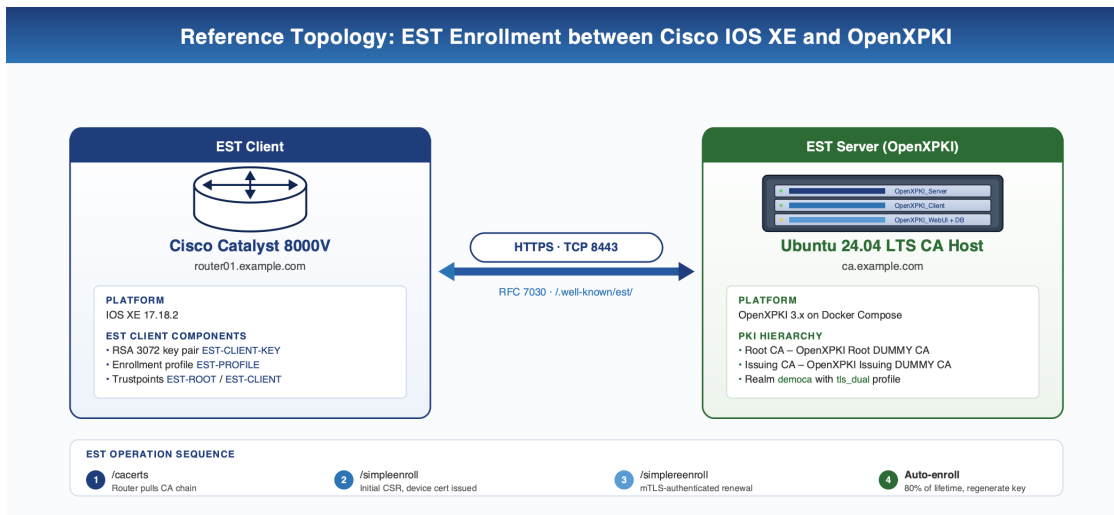
Chapter	Content
	and Apache vhost.
Chapter 6 - IOS XE EST Client Configuration	End-to-end IOS XE configuration and enrollment procedure.
Chapter 7 - Testing and Verification	Sanity checks from the CA host, from OpenSSL, and from the router.
Chapter 8 - Troubleshooting and Common Issues	Every failure mode encountered during the lab build and its resolution.
Chapter 9 - Day-Two Operations	Reload, renewal, log locations, and backup.

In addition to the chapter text, the guide ships with a set of reference configuration snippets (OpenXPki YAML under `examples/openxpki/` and IOS XE running-config fragments under `examples/iosxe/`) and a handful of helper scripts (`scripts/`) that were used to drive interactive prompts on the router during the lab build. These are surfaced at the relevant point inside each chapter.

Quick start

1. Review Chapter 1 (Prerequisites) and verify DNS, NTP, and reachability between the CA host and the IOS XE device.
2. Follow Chapter 2 (OpenXPki Installation on Ubuntu) to bring up the OpenXPki Docker stack.
3. Follow Chapter 3 (Bootstrapping the PKI) to create the Root and Issuing CAs.
4. Follow Chapter 4 (TLS Server Certificate for the EST Endpoint) to issue the EST front-end TLS server certificate.
5. Follow Chapter 5 (EST Endpoint Configuration in OpenXPki) to turn on the EST endpoint.
6. Follow Chapter 6 (IOS XE EST Client Configuration) on the router.
7. Use Chapter 7 (Testing and Verification) to validate the issued certificate.
8. When anything fails, consult Chapter 8 (Troubleshooting and Common Issues).

Reference topology



Reference topology: EST enrollment between a Cisco IOS XE router and an OpenXPKI CA over HTTPS/TCP 8443.

Versions tested

- Ubuntu 24.04 LTS
- Docker Engine 27.x with Docker Compose v2
- OpenXPKI Community Edition 3.x (image tag published by the openxpki-docker project at time of writing)
- Cisco Catalyst 8000V running IOS XE 17.18.2

IOS XE 17.18 is the recommended minimum. Earlier trains also support EST, but some of the TLS handshake behavior and method-est ergonomics we rely on in this guide are only consistent on 17.18+.

Security posture of the lab

The configuration produced by this guide is tightened along the practical axes an operator cares about, with the lab-specific shortcuts clearly scoped:

Already tightened (and verified end-to-end in the lab):

- `authorized_signer.rule1` is pinned to profile: `tls_dual` plus a subject regex (`.*DC=Test Deployment,DC=OpenXPKI,DC=org`) so that only certs issued under the fleet enrollment profile can renew themselves on `/simplereenroll`. Replaying a cert issued under a different profile (for example a WebUI `user_auth_enc` cert) is rejected. See Chapter 5 section 1.
- The `tls_dual` profile enforces an FQDN allow-list on both the CN and every DNS SAN - a CSR for `evil.example.com` is refused with an empty-DN error even if the caller can authenticate. See Chapter 5 section 2.

- Every issued certificate carries an explicit CDP pointing at the CA's CRL download URL, plus SKI/AKI. See Chapter 3 section 8.
- The WebUI vhost ships with HSTS, X-Frame-Options: DENY, nosniff, Referrer-Policy, Permissions-Policy, and a first-party CSP. See Chapter 5 section 3c.
- /simpleenroll is protected by per-device HTTP Basic bootstrap credentials in a bcrypt-hashed httpasswd file, with a documented rotation cadence and a two-hash rollover helper (scripts/rotate-est-bootstrap-cred.sh). Post-bootstrap, the device does not re-present the credential - renewal runs over mTLS on /simpleenroll using the device's current certificate. See Chapter 5 sections 3b and 3d.
- The router trustpoint uses auto-enroll N regenerate, so each renewal submits a freshly minted key pair - a key compromise is bounded to one certificate's lifetime. See Chapter 6.
- revocation-check crl none on the router's EST-CLIENT trustpoint pulls the published CRL and caches it, falling back to the none fail-open only if the CA's CRL endpoint is unreachable. See Chapter 6.

Deliberately lab-scoped (swap before production cutover):

- The democa realm, Root CA, and Issuing CA are labeled DUMMY CA and were bootstrapped from OpenXPKI's sampleconfig.sh. That script seeds test data and is not a production-grade bootstrap; replace the hierarchy before real onboarding.
- The CA's signing key sits in the software svault token. For production, migrate the CA key into an HSM / KMS via OpenXPKI's PKCS#11 token support.
- The CRL download endpoint is served over plain HTTP on port 8080 (CRLs are already signed, so HTTP is acceptable by design, but putting the endpoint behind a hardened reverse proxy or CDN is recommended).
- The EST-ROOT trustpoint on the router uses revocation-check none because a self-signed Root has no meaningful revocation path.

Chapter 9 (Day-Two Operations) keeps the consolidated hardening checklist and flags every item that is still open for a genuine production cutover.

Chapter 1. Prerequisites

This chapter documents everything that must be true before you begin installing OpenXPki or touching the router. If you skip any of these, you will hit one of the failure modes in Chapter 8.

Hosts

CA host (Ubuntu)

- OS: Ubuntu 22.04 LTS or 24.04 LTS, 64-bit.
- CPU: 2 vCPU minimum; 4 vCPU recommended.
- RAM: 4 GB minimum; 8 GB recommended.
- Disk: 20 GB free in `/var/lib/docker` (the Docker image and the MariaDB volume both live here).
- User: a non-root account with `sudo` privileges. Passwordless `sudo` simplifies automation but is not required.
- Time: NTP synchronized and monotonically correct. Certificate validity windows are measured in seconds; a 10-minute clock skew will cause TLS and PKI operations to fail.

IOS XE device

- IOS XE 17.18.x or newer (earlier trains work but have quirks around `method-est` and HTTP Basic authentication).
- `ip http client` globally enabled (required even for HTTPS EST on some images).
- Time synchronized to a reachable NTP source. The router cannot enroll if its clock is outside the CA certificate validity window.
- DNS resolver configured or static `ip host` entry for the CA FQDN. Hostname verification is mandatory - EST only accepts certificates whose CN/SAN matches the URL.
- An RSA key pair dedicated to EST (EST-CLIENT-KEY in this guide). 3072 bits is the current NIST recommendation and what this guide uses throughout. 2048 bits is still acceptable for short-lived certs but is being phased out. ECDSA on P-256 or P-384 is an equally valid alternative where the whole ecosystem supports it.

Networking

- The router must be able to reach the CA host on TCP/8443.
- The CA host must resolve its own FQDN to the IP address that the router will contact. Apache and the issued TLS certificate both key off this FQDN.
- Plan to use a single FQDN for the EST service. Do not mix “10.x.x.x” in the IOS XE `enrollment url` with an FQDN in the TLS certificate - IOS XE will fail hostname verification.

DNS

Pick one FQDN for the CA host and use it everywhere: - In the TLS certificate CN and SAN (DNS). - In the IOS XE enrollment url on the EST profile. - In any firewall rule or allow list.

In this guide the FQDN is `svs-rtp-ca2.cisco.com`. Adjust to your environment.

Quick DNS sanity check from the router:

```
pgiralt-c8kv-test#ping svs-rtp-ca2.cisco.com
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.122.249.20, timeout is 2 seconds:
!!!!
```

NTP

```
pgiralt-c8kv-test#show clock detail
20:53:26.427 UTC Wed Apr 22 2026
Time source is NTP
```

If the router cannot reach an NTP source, configure `clock set` manually for the lab but understand that any future reload will lose the time and break EST.

On the CA host:

```
administrator@svs-rtp-ca2:~$ timedatectl | grep -i 'System clock synchronized'
System clock synchronized: yes
```

Operating-system packages on the CA host

Required: - `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin`, `docker-compose-plugin` (or the matching Ubuntu-provided versions). - `openssl 3.x` for generating CSRs and inspecting certificates. - `git` for cloning the `openxpk-docker` repository. - `curl` for sanity checks against the EST endpoints.

Installation of Docker Engine on Ubuntu is covered in Chapter 2.

SSH access model used in this guide

The guide assumes you have key-based SSH access to both hosts:

```
ssh administrator@svs-rtp-ca2.cisco.com # CA host, Ubuntu
ssh pgiralt@pgiralt-c8kv-test.cisco.com # Router, IOS XE
```

Terminal commands in this guide are shown with their prompts so you know which host to run them on.

Firewall and security considerations

- Expose TCP/8443 only on the interfaces the router actually needs. Restrict source IPs at the network boundary.
- TCP/8080 on the OpenXPKI WebUI serves the HTTP redirect and ACME. Do not expose it to the public Internet.
- TCP/3306 (MariaDB) is bound inside the Docker network only.

Hardening notes (for later)

The configuration walked through in this guide already implements the `authorized_signer` tightening, the FQDN allow-list on the `tls_dual` profile, per-device HTTP Basic bootstrap credentials, mTLS-based renewal, CRL publication, and the full set of security headers on the WebUI vhost. Chapter 9 (Day-Two Operations) keeps the consolidated checklist.

The items still open for a genuine production cutover are:

- Build a real two-tier PKI with the CA signing key generated on an HSM/KMS, not the `sampleconfig.sh` output. See OpenXPKI documentation for the production bootstrap workflow.
- Put rate limiting in front of Apache (`mod_security` or an upstream WAF, plus `fail2ban` on the `est_access.log` 401s).
- Ship server, client, audit, and Apache logs to a central SIEM with structured fields (OpenXPKI already writes JSON-compatible logs).
- Put the CRL download endpoint (TCP/8080) behind a hardened reverse proxy or CDN. CRLs are already signed so plain HTTP is acceptable by design, but the server itself should not be directly Internet-facing without rate limiting.

Chapter 2. OpenXPKI Installation on Ubuntu

This chapter walks through installing Docker Engine and deploying OpenXPKI 3.x using the upstream `openxpi-docker` project. At the end you will have four OpenXPKI containers running (Database, Server, Client, WebUI) and TCP/8443 serving the OpenXPKI web interface.

1. Install Docker Engine

Install Docker Engine from the official Docker APT repository so that you get a current version and the Compose v2 plugin. On Ubuntu 24.04:

```
# On the CA host, as the admin user
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list >/dev/null

sudo apt-get update
sudo apt-get install -y \
    docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
```

Verify:

```
docker version
docker compose version
```

Add your user to the docker group so you can drive the stack without sudo, then log out and back in:

```
sudo usermod -aG docker "$USER"
```

2. Clone the openxpi-docker repository

The upstream project is at <https://github.com/openxpi/openxpi-docker>. This guide places everything under `~/est-lab`:

```
mkdir -p ~/est-lab && cd ~/est-lab
git clone https://github.com/openxpi/openxpi-docker.git
```

The repository contains:

- `docker-compose.yml` - four-service stack (db, server, client, web).
- `openxpki-config/` - seed configuration that becomes `/etc/openxpki/` inside the containers.
- `openxpki-config/contrib/apache2-openxpki-site.conf` - the Apache vhost template that the web container mounts. You will edit this for EST.

3. Prepare the datavault secret

OpenXPki encrypts the datavault with a symmetric secret named `svault`. The seed config has a placeholder that must be replaced before the stack starts.

Generate a strong 64-hex-character key and write it into the crypto config:

```
cd ~/est-lab
SVAULTKEY=$(openssl rand -hex 32)
sed -i "s/##SVAULTKEY##/$SVAULTKEY/" \
  openxpki-config/config.d/system/crypto.yaml
grep -n '^          value:' openxpki-config/config.d/system/crypto.yaml
```

The resulting block looks like:

```
secret:
  svault:
    label: Secret group for datavault encryption
    method: literal
    value: db6a20b0a4a61e385e75fb0b6a2b59f13a5fb0428eeb260401c0bf55fbf70e1a
```

Security note: the `svault` key is the root of the OpenXPki confidentiality model. In production, store it in a secret manager and inject it at container start, not committed to a repository.

4. Configure the oxi CLI authentication

OpenXPki's CLI (`oxi`) authenticates to the server with an EC key pair. The default seed config leaves this empty. Generate a key and configure the admin user:

```
cd ~/est-lab
openssl ecparam -name prime256v1 -genkey -noout -out /tmp/oxi-admin.key
openssl ec -in /tmp/oxi-admin.key -pubout -out /tmp/oxi-admin.pub
cat /tmp/oxi-admin.pub
```

Install the private key into the admin user's home (on the host, to be used from the container via a volume mount, or simply copied in):

```
mkdir -p ~/.oxi
cp /tmp/oxi-admin.key ~/.oxi/admin.key
chmod 600 ~/.oxi/admin.key
```

Edit `openxpki-config/config.d/system/cli.yaml` and add the public key with a role. For lab use, RA Operator is a convenient role because it can manage pending enrollment workflows:

```
auth:
  admin:
    key: |
      -----BEGIN PUBLIC KEY-----
      MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEFG9msov1/7FPVyZV5ZMqOqcMU755
      qsUcf/YVCR5XtElfEB58wlyBK+ov6WuAeLvLG6+wKumUkn/N9m+073gtQg==
      -----END PUBLIC KEY-----
    role: RA Operator
```

See `examples/openxpki/cli.yaml` for a ready-to-copy reference.

5. Start the stack

```
cd ~/est-lab/openxpki-docker
docker compose up -d
docker compose ps
```

Expected output (abbreviated):

NAME	COMMAND	STATE	PORTS
OpenXPKI_Database	"docker-entrypoint.s..."	healthy	3306/tcp
OpenXPKI_Server	"/usr/bin/start-serv..."	healthy	
OpenXPKI_Client	"/usr/bin/start-clie..."	healthy	
OpenXPKI_WebUI	"/usr/bin/start-webs..."	healthy	0.0.0.0:8080->80/tcp, 0.0.0.0:8443->443/tcp

If any service is unhealthy, jump to Chapter 8.

6. Smoke test the web front-end

```
curl -sk https://localhost:8443/healthcheck/ping
# -> "OK" (or JSON status object)
```

Browse to `https://ca.example.com:8443/` and you should see the OpenXPKI WebUI. At this point there is no PKI inside yet; that's the next chapter.

7. Useful operational commands

```
# Tail all service logs
docker compose logs -f

# Reload server config after editing YAML (no downtime)
docker exec OpenXPKI_Server openxpkitl reload server
docker exec OpenXPKI_Client openxpkitl reload client

# Restart just the web tier (required after Apache config changes)
```

```
docker restart OpenXPki_WebUI
```

```
# Restart the full stack
```

```
docker compose restart
```

8. What the web container mounts

The Compose file mounts several host paths read-only into the web container:

```
web:
```

```
  volumes:
```

```
    - ./openxpki-config/contrib/apache2-openxpki-site.conf:/etc/apache2/sites-enabled/openxpki.conf
    - ./openxpki-config/tls:/etc/openxpki/tls/
    - openxpki-clientsocket:/run/openxpki-clientd
```

This matters for later chapters:

- Apache vhost changes are edited on the host and picked up by restarting the container.
- TLS server certificates live under `openxpki-config/tls/` and are consumed directly by Apache.

Next: Chapter 3.

Chapter 3. Bootstrapping the PKI

This chapter creates the CA hierarchy inside OpenXPKI using the bundled sample script. At the end you will have a Root CA, an Issuing CA, and a ready democa realm suitable for issuing the EST server certificate and device certificates.

1. Decide on labels and validity

For the lab we use the default labels produced by the sample script, which are of the form:

- OpenXPKI Root DUMMY CA 20260422 - 10 years
- OpenXPKI Issuing DUMMY CA 20260422 - 10 years, signed by the Root

If you intend to keep this environment around, edit the sample script before running it to remove the word DUMMY and set the validity you want. The script is idempotent-enough to rerun with `--force`, but it is easier to pick sensible names up front.

2. Run the sample-config script

The sample script lives at `/usr/share/doc/libopenxpki-perl/examples/sampleconfig.sh` inside the `OpenXPKI_Server` container. It generates keys, builds the certificate chain, imports everything into the OpenXPKI database, and assigns the tokens to the democa realm.

```
# On the CA host
docker exec OpenXPKI_Server \
  /usr/share/doc/libopenxpki-perl/examples/sampleconfig.sh
```

Expected output ends with:

```
== Checking database connectivity
Database check passed
== Import generated certificates
== Assign tokens
== Done
```

What it does behind the scenes:

- Generates an ECDSA P-384 key for each CA.
- Creates a self-signed Root CA certificate.
- Creates an Issuing CA CSR, signs it with the Root, and imports both.
- Generates dedicated tokens (`ca-signer-1`, `vault-1`, `scep-1`) and aliases them in the democa realm.
- Issues an initial CRL so that revocation infrastructure is in place.

After the script finishes, reload the server so the new tokens are active:

```
docker exec OpenXPKI_Server openxpkitl reload server
```

3. Verify the realm

Still on the CA host:

```
docker exec OpenXPKI_Server oxi api list_alias \
  --realm democa --group certsign
```

You should see the Issuing CA alias (ca-signer-1) with its serial and validity. If you do not see it, go back and rerun the sample script - the most common cause is that the database connection was not yet ready when the script tried to import.

4. Confirm certificates in the database

```
docker exec OpenXPKI_Server oxi api search_cert \
  --realm democa --subject "cn=OpenXPKI%"
```

You should see two entries:

- cn=OpenXPKI Root DUMMY CA 20260422 - self-issued.
- cn=OpenXPKI Issuing DUMMY CA 20260422 - issued by the Root.

5. Check the web UI

Log into <https://ca.example.com:8443/> and switch to the democa realm in the top-right picker. The dashboard should now show the CA hierarchy and recent workflows.

6. Export the Root CA for IOS XE

You will need the Root CA in PEM form later so that IOS XE can authenticate the chain. Extract it from the database now so you have it to hand:

```
docker exec OpenXPKI_Server oxi api get_cert \
  --identifier $(docker exec OpenXPKI_Server oxi api list_alias \
    --realm democa --group root | awk '/identifier/ {print $2; exit}') \
  --format PEM > ~/est-lab/root-ca.pem
cat ~/est-lab/root-ca.pem | openssl x509 -noout -subject -issuer
```

Expected:

```
subject=CN = OpenXPKI Root DUMMY CA 20260422, OU = PKI, O = OpenXPKI, C = DE
issuer=CN = OpenXPKI Root DUMMY CA 20260422, OU = PKI, O = OpenXPKI, C = DE
```

Keep root-ca.pem - it's imported verbatim into the router in Chapter 6.

7. (Optional) Trust the Root on your workstation

So that your browser can reach the WebUI without TLS warnings once the CA-signed front-end certificate is in place:

- macOS: open `root-ca.pem` with Keychain Access and mark it as trusted for SSL.
- Linux: `sudo cp root-ca.pem /usr/local/share/ca-certificates/openxpki-root.crt && sudo update-ca-certificates.`
- Windows: `certutil -addstore -f "ROOT" root-ca.pem.`

8. Set the CRL Distribution Point (CDP) URL

By default the sample profile writes a placeholder CDP URL (`http://pki.example.com/...`) into every certificate it issues. That URL is useless to a real client and causes CRL-check failures the first time anyone turns on revocation checking. Point it at the host where the CA will publish the CRL (same FQDN as the EST endpoint in this lab).

Edit `~/est-lab/openxpki-docker/openxpki-config/config.d/realm/democa/profile/default.yaml` and replace the entire `crl_distribution_points` block with:

```
# CRL Distribution Point (CDP) - points at the CA's own HTTP  
# download endpoint (port 8080 inside the Lab; non-TLS by design so  
# that clients verifying the CRL do not need a second TLS handshake  
# just to fetch it). The template expands [% ISSUER.CN.0 %] to the  
# Issuing CA's CN so that each generation of the Issuing CA publishes  
# under its own filename (e.g. OpenXPKI_Issuing_DUMMY_CA_20260422.crl).  
crl_distribution_points:  
  critical: 0  
  uri:  
    - http://ca.example.com:8080/download/[% ISSUER.CN.0.replace(' ','_') %].crl
```

While you are in this file, also delete the `authority_info_access` block and the `policy_identifier` block. Both ship with placeholder `example.com` URLs that would otherwise embed into every issued cert - pointing to a policy document and an OCSP responder that do not exist. You can re-enable them later when you stand up a real OCSP responder and a Certificate Policy document.

Reload so OpenXPki picks up the new profile:

```
docker exec OpenXPki_Server openxpkitl reload server  
docker exec OpenXPki_Client openxpkitl reload client
```

Verify by issuing a test certificate and dumping the CDP extension:

```
openssl x509 -in <any-newly-issued-cert>.pem -noout -ext crlDistributionPoints
```

Expected (after the change):

X509v3 CRL Distribution Points:

Full Name:

URI: `http://ca.example.com:8080/download/OpenXPki_Issuing_DUMMY_CA_20260422.crl`

This URL is consumed by `revocation-check crl` on the router's trustpoint (see Chapter 6). In this lab the port is 8080, plain HTTP; that is intentional. Fetching a CRL over a CA-validated TLS channel is circular (you would need the CRL to validate the server cert that is serving the CRL). Industry practice is to serve CRLs over plain HTTP and rely on the CRL's own CMS signature, which the router verifies against the Issuing CA certificate it already trusts.

9. What we did not do

- We did not move the CA keys into an HSM or KMS. The sample script keeps them on disk under `/etc/openxpki/ca/`. This is a lab profile.
- We did not stand up an OCSP responder. OpenXPki supports one as a separate endpoint; this guide uses CRLs instead to keep the moving parts minimal.

Next: Chapter 4 to issue the EST endpoint's own TLS certificate.

Chapter 4. TLS Server Certificate for the EST Endpoint

Before IOS XE can talk to the EST endpoint, the front-end Apache must present a certificate that chains to the OpenXPki Issuing CA and has the CA hostname in both the Common Name and a subjectAltName DNS entry. IOS XE performs RFC 6125 hostname verification and will refuse to continue if either is missing.

1. Generate a key and CSR on the CA host

Pick a dedicated working directory so the material does not collide with other files:

```
mkdir -p ~/est-lab/web-cert
cd ~/est-lab/web-cert
```

Write a small OpenSSL config that sets the CN and SAN:

```
cat > san.cnf <<'EOF'
[req]
distinguished_name = dn
prompt              = no
req_extensions      = req_ext
[dn]
CN = svcs-rtp-ca2.cisco.com
[req_ext]
subjectAltName = DNS:svcs-rtp-ca2.cisco.com, IP:10.122.249.20
EOF
```

Generate a 3072-bit RSA key and a CSR:

```
openssl req -new -nodes \
  -newkey rsa:3072 \
  -keyout web.key -out web.csr \
  -config san.cnf
```

Verify the CSR carries the SAN extension:

```
openssl req -in web.csr -noout -text | grep -A1 "Subject Alternative Name"
```

Expected:

```
X509v3 Subject Alternative Name:
  DNS:svcs-rtp-ca2.cisco.com, IP Address:10.122.249.20
```

2. Submit the CSR via EST (eat your own dog food)

You can issue this certificate via the OpenXPki WebUI if you prefer, but because we are building an EST environment it's instructive to issue the CA's own server certificate via the `/simpleenroll` endpoint on localhost. This also proves that the `tls_server` (or `tls_dual`) profile is working.

```

# Convert the CSR to base64-encoded DER for EST
openssl req -in web.csr -outform DER | base64 > web.b64

# Fetch the current CA chain from the EST endpoint so we trust ourselves
mkdir -p ~/est-lab/est-test
curl -sk https://svs-rtp-ca2.cisco.com:8443/.well-known/est/cacerts \
  -o ~/est-lab/est-test/cacerts.p7
openssl pkcs7 -in ~/est-lab/est-test/cacerts.p7 \
  -inform PEM -print_certs > ~/est-lab/est-test/ca.pem 2>/dev/null || \
  openssl pkcs7 -in ~/est-lab/est-test/cacerts.p7 \
    -inform DER -print_certs > ~/est-lab/est-test/ca.pem

# Post the CSR
curl -sk --cacert ~/est-lab/est-test/ca.pem \
  https://svs-rtp-ca2.cisco.com:8443/.well-known/est/simpleenroll \
  --data-binary @web.b64 \
  -H "Content-Type: application/pkcs10" \
  -H "Content-Transfer-Encoding: base64" \
  -o web.p7 -w "HTTP=%{http_code}\n"

```

At this point in the chronological build-out, the EST endpoint still accepts anonymous /simpleenroll requests - Chapter 5 is where we add Basic authentication at Apache. If you are re-running this command after the endpoint has been locked down, append -u est-user:EstLabPass2026 (adjust the credential to match your htpasswd/est.passwd).

If you get HTTP=200, you have a PKCS#7 bundle in web.p7. If you get 202 ("Pending") or an error, jump to Chapter 8.

Extract the leaf certificate:

```

openssl pkcs7 -inform DER -in web.p7 -print_certs -out web-chain.pem 2>/dev/null || \
  openssl base64 -d -A -in web.p7 | \
  openssl pkcs7 -inform DER -print_certs -out web-chain.pem

# web-chain.pem may contain multiple certs; take only the leaf as web.crt
awk 'BEGIN{n=0} /-----BEGIN CERTIFICATE-----/{n++; n==1{print}}' \
  web-chain.pem > web.crt

```

Verify:

```
openssl x509 -in web.crt -noout -subject -issuer -ext subjectAltName
```

Expected:

```

subject=CN = svs-rtp-ca2.cisco.com
issuer=CN = OpenXPki Issuing DUMMY CA 20260422
X509v3 Subject Alternative Name:
  DNS:svs-rtp-ca2.cisco.com, IP Address:10.122.249.20

```

3. Install the certificate into Apache

The web container mounts the host path `~/est-lab/openxpki-config/tls/` as `/etc/openxpki/tls/`. Copy the key, the leaf-plus-issuer chain, and the Root into the matching slots:

```
cd ~/est-lab
# Private key
install -m 600 web-cert/web.key openxpki-config/tls/private/openxpki.pem

# Leaf + issuing CA chain in one file
cat web-cert/web.crt > openxpki-config/tls/ententity/openxpki.crt
# Append the Issuing CA cert so Apache can serve the full chain
openssl pkcs7 -in est-test/cacerts.p7 -inform DER -print_certs 2>/dev/null | \
    awk '/-----BEGIN/{flag=1} flag' \
    >> openxpki-config/tls/ententity/openxpki.crt

# Trust anchor (Root CA) in the chain dir for mTLS client verification
cp root-ca.pem openxpki-config/tls/chain/root.crt
c_rehash openxpki-config/tls/chain/ 2>/dev/null || \
    openssl rehash openxpki-config/tls/chain/
```

Then restart the web container:

```
docker restart OpenXPki_WebUI
```

4. Validate the TLS endpoint

From the CA host:

```
openssl s_client -connect svb-rtp-ca2.cisco.com:8443 \
    -servername svb-rtp-ca2.cisco.com -showcerts < /dev/null 2>&1 | \
    grep -E 'subject=|issuer=' | head -6
```

Expected:

```
0 s:CN = svb-rtp-ca2.cisco.com
  i:CN = OpenXPki Issuing DUMMY CA 20260422
1 s:CN = OpenXPki Issuing DUMMY CA 20260422
  i:CN = OpenXPki Root DUMMY CA 20260422
```

From the router:

```
pgiralt-c8kv-test#telnet svb-rtp-ca2.cisco.com 8443
Trying svb-rtp-ca2.cisco.com (10.122.249.20, 8443)... Open
```

If either of these fails, revisit the SAN configuration or check that the container actually reloaded with the new certificate.

5. What `tls_server` vs `tls_dual` gives you

OpenXPKI ships two relevant certificate profiles:

- `tls_server` - only `serverAuth` EKU. Fine for this TLS cert if the hostname will never be used as a client.
- `tls_dual` - both `serverAuth` and `clientAuth` EKU. Required for the IOS XE *device* certificate, because the router uses the same identity cert as a TLS *client* in several places:
 - `/simplereenroll` against this EST endpoint, which is mutually authenticated (see Chapter 5).
 - TACACS+ over TLS (RFC 8907 Appendix C / draft-ietf-opsawg-tacacs-tls13) where the router is the TLS client talking to the AAA server.
 - RADIUS/TLS, a.k.a. RadSec (RFC 6614), where the router is the TLS client talking to the RADIUS server.
 - Any future feature that does outbound mTLS from the device (NETCONF over TLS with client-cert auth, SYSLOG/TLS with mutual auth, pxGrid, etc.).

If the device certificate is issued with only `serverAuth`, every one of those features either fails the TLS handshake or silently falls back to username/password. This is one of the most common footguns in EST deployments, so treat `clientAuth` as non-optional on the device profile.

The EST *front-end* TLS certificate (what this chapter issues) is, by contrast, only ever used as a server and could technically be issued from `tls_server`. In the reference lab we use `tls_dual` end-to-end for both the front-end and the devices so there is a single enrollment profile to reason about and rotate; see Chapter 5.

6. Rotation

When the TLS certificate expires, simply rerun the EST enrolment against a fresh CSR (same SAN) and reinstall. OpenXPKI will auto-revoke the previous certificate if `policy.auto_revoke_existing_certs` is set in the endpoint profile.

Next: Chapter 5 to enable the EST endpoint itself.

Chapter 5. EST Endpoint Configuration in OpenXPKI

This chapter wires up the OpenXPKI EST endpoint and makes Apache forward the right traffic. It covers:

- The `est/default.yaml` policy file for the democa realm.
- The `tls_dual` certificate profile.
- Apache configuration, including the split between `/simpleenroll` (HTTP Basic authentication for initial bootstrap) and `/simplereenroll` (mutual TLS with the device's current certificate for renewal).

Credential model used in this chapter

Two authentication styles coexist on the same EST endpoint - one for the first-time bootstrap, one for every renewal afterwards:

EST operation	When	Authentication	Enforced by
<code>/cacerts</code>	Before every operation	Anonymous	Nothing
<code>/simpleenroll</code>	Initial bootstrap only	HTTP Basic	Apache AuthType Basic + htpasswd
<code>/simplereenroll</code>	Every renewal	mTLS with current device cert	Apache +OptRenegotiate + SSLVerifyClient

The YAML and Apache snippets in the next sections are the server-side half of that model. The matching IOS XE configuration - which credential the router sends for which operation, and why the two coexist safely - is in Chapter 6 section 4a. If the lifecycle view in the README's "How the device authenticates across its lifetime" section did not stick, re-read that before continuing.

1. Endpoint policy (`est/default.yaml`)

File: `~/est-lab/openxpki-config/config.d/realm/democa/est/default.yaml`

Reference configuration:

label: EST Default Endpoint

authorized_signer:

```
# Who is allowed to present a client certificate on /simplereenroll?
# (This block does NOT apply to /simpleenroll, which has no client
# cert and is authenticated at the edge by Apache HTTP Basic.)
#
# The default tpl shipped with OpenXPKI is `subject: ".*"`, which
# would accept any cert signed by the Issuing CA, including certs
# issued under different profiles (e.g. a user_auth_enc cert being
# replayed to the EST endpoint). We tighten that in two dimensions:
#
# - `profile` must be `tls_dual` - the fleet enrollment profile
```

```
# used by /simpleenroll. Certs issued under any other profile
# in this realm (e.g. a WebUI user_auth_enc cert) cannot be
# replayed here to renew themselves into a TLS client cert.
# OpenXPKI resolves profile by looking the signer cert up in
# the certificate DB, so this also implicitly pins the issuer
# to this CA - an externally-signed cert will not match.
#
# - `subject` must match the fleet DN template. The tls_dual
# `enroll` style always appends DC=Test Deployment,DC=OpenXPKI,
# DC=org, so anything with a different DN shape is rejected
# here as defense in depth. The match is anchored (^..$) by
# OpenXPKI before being run against the signer subject.
#
# NOTE: do NOT use `issuer_dn` here - it is not a recognized rule
# key and silently forces the rule to fail (see the OpenXPKI
# evaluate_trust_rule module). Use `profile` (as here),
# `issuer_alias`, or `root_alias` instead.
```

rule1:

```
profile: tls_dual
subject: ".*DC=Test Deployment,DC=OpenXPKI,DC=org"
```

```
# How far from notAfter OpenXPKI accepts /simplereenroll. Format is DDDhh
# (days and hours) where "000060" = 60 days, "010000" = 365 days.
```

```
#
# For a production deployment with 1-year certs and auto-enroll 80, a
# 60-day window is a reasonable default (the router renews roughly
# 73 days before expiry).
```

```
#
# For a Lab that issues short-lived certs or renews on demand, widen the
# window so every in-validity request is accepted:
```

```
# renewal_period: "010000"
```

```
renewal_period: "010000"
```

policy:

```
# Trust what Apache Lets through. Apache requires HTTP Basic auth on
# /simpleenroll (initial bootstrap) and mTLS on /simplereenroll, so
# by the time OpenXPKI sees the request the caller has already been
# authenticated at the edge. Setting this to 0 without also forwarding
# REMOTE_USER to the backend would make OpenXPKI reject every request
# as anonymous.
```

```
allow_anon_enroll: 1
```

```
# Let an RA Operator approve any outlier requests manually in the WebUI.
```

```
allow_man_approv: 1
```

```
# Keep a single active cert per subject and revoke the previous one
# when a new one is issued.
```

```
max_active_certs: 1
```

```
auto_revoke_existing_certs: 1
```

```
# Auto-approve once eligibility is confirmed (see eligible.* below).
```

```
approval_points: 0
```

profile:

```
cert_profile: tls_dual
cert_subject_style: enroll
```

```
eligible:
  initial:
    # 1 = auto-grant eligibility for an initial enroll request. This
    # is what makes fully automated bootstrap work. Apache's Basic-auth
    # check at the edge is what restricts *who* can reach this endpoint.
    value: 1
  renewal:
    value: 1
  onbehalf:
    # Disable on-behalf enrollment to prevent OpenXPki from treating the
    # router's presented self-signed cert as a signer during bootstrap.
    value: 0
```

Key points about the authentication model:

- OpenXPki itself is configured to trust the caller (`allow_anon_enroll: 1`). The decision to accept or reject a bootstrap request is made *at the edge*, by Apache, using HTTP Basic authentication. Flipping this flag to 0 without also wiring up `REMOTE_USER/X-ReverseProxy-ENV-...` forwarding on the proxy would make OpenXPki reject every request with “not eligible” because it cannot see the Basic-auth credential that Apache already consumed.
- `approval_points: 0` tells the workflow engine that no manual approval step is required to issue the certificate.
- `eligible.initial.value: 1` is the critical second half of the auto-approval story. Both flags must be true to fully auto-approve a CSR that reaches the backend.
- `eligible.onbehalf.value: 0` disables the “on-behalf” path. If you leave it enabled, OpenXPki will try to validate the router’s presented cert as an RA signer cert on `/simplereenroll` and reject with `SIGNER_NOT_AUTHORIZED`.

A copy-paste copy lives at `examples/openxpki/est-default.yaml`.

2. The `tls_dual` certificate profile

The default `tls_dual` profile (`realm/democa/profile/tls_dual.yaml`) emits both `serverAuth` and `clientAuth` EKUs. Both are mandatory on an IOS XE identity certificate: the router is a TLS *server* for things like HTTPS management and NETCONF over TLS, and a TLS *client* for `/simplereenroll`, TACACS+ over TLS, RADIUS/TLS (RadSec), and similar outbound mTLS use cases. Issuing only `serverAuth` here is a common footgun that breaks those features after bootstrap; see Chapter 4 for the longer rationale.

On top of the shipped defaults the profile needs three production hardening changes:

1. An `enroll` subject style with an FQDN allow-list, so a device that somehow reaches the endpoint cannot enroll as `evil.example.com` (item 4).
2. Explicit `subject_key_identifier / authority_key_identifier` blocks so those extensions cannot be lost if someone later overrides extensions: in a parent profile (item 9).

3. A short validity (47 days, tracking the CA/B Forum direction for TLS certs).

Full reference file - copy this to realm/democa/profile/tls_dual.yaml:

label: TLS Server / Client Certificate

style:

enroll:

label: Enroll with CN and SAN from CSR

*# Fleet allow-list for automated enrollment (EST /simpleenroll and
/simplereenroll). The DN is only built when the CN is either:*

- A bare short hostname (no dot), or

- A fully-qualified name ending in .ecatslab.com

Anything else (e.g. a device that tries to enroll as

"evil.example.com") yields an empty DN, which OpenXPKI refuses

to sign with a clear subject-validation error. The same

allow-list is applied to every DNS SAN; disallowed entries are

dropped so a malicious CSR cannot smuggle a rogue SAN through

even if the CN happens to match.

#

To support additional fleets, change the regex in both the

`dn:` line and the `dns:` line below.

subject:

dn: "[% IF CN.0.match('^[A-Za-z0-9-]+|[A-Za-z0-9-]+\.\.ecatslab\.\.com\$') %]CN=[%
CN.0 %],DC=Test Deployment,DC=OpenXPKI,DC=org[% END %]"

san:

dns: "[% FOREACH entry = SAN_DNS %][% IF entry.match('^[A-Za-z0-9-]+|[A-Za-z0-9-
-]+\.\.ecatslab\.\.com\$') %][% entry.lower %] | [% END %][% END %]"

ip : "[% FOREACH entry = SAN_IP %][% entry %] | [% END %]"

extensions:

key_usage:

critical: 1

digital_signature: 1

key_encipherment: 1

extended_key_usage:

critical: 0

client_auth: 1

server_auth: 1

*# Explicitly declare SKI/AKI here so the intent is obvious at the
profile level and the extensions are not lost if someone later
overrides the `extensions:` block in default.yaml. SKI is the
SHA-1 hash of the public key (used by the router to index certs
in nvram); AKI (keyid) pins this cert to the Issuing CA that
signed it, providing path-building continuity across renewals.*

subject_key_identifier:

critical: 0

hash: 1

authority_key_identifier:

critical: 0

```

keyid: 1
issuer: 0

subject_alt_name:
  critical: 0
  copy: 1

```

*# Validity override: 47 days, tracking the CA/B Forum direction for
short-lived TLS certs. With auto-enroll 80 regenerate on the router this
means a fresh cert roughly every 37 or 38 days.*

```

validity:
  notafter: "+000047"

```

Three pieces worth calling out:

- `subject_alt_name.copy: 1` keeps whatever SANs the CSR supplied. The `enroll` style above then filters those SANs through the allow-list. Both mechanisms run; the stricter one wins.
- The FQDN regex is anchored (`^...$`). A bare hostname matches because `[A-Za-z0-9-]+` has no dot alternative; an FQDN matches only when it ends in `.ecatslab.com`. Change the domain in **both** the `dn` line and the `dns` line when onboarding additional fleets - missing one side is a silent bypass.
- The OpenXPki `notafter` relative format is `+YYYYMMDD` (or `+YYYYMMDDhhmmss`). `+000047` means “now + 47 days”. A common misread is `+00000047`, which actually means “now + 47 hours”.

Apply it:

```

docker compose -f ~/est-lab/openxpki-docker/docker-compose.yml \
  restart server client

```

Profile changes often need a full restart - a `reload` is not always enough for profile template evaluation caches.

Verify the allow-list

Because OpenXPki deduplicates by CSR hash, you have to use a *new* key each time you test the allow-list - an already-rejected CSR will be pulled from cache and rejected again with a stale error. Two quick tests:

```

# GOOD: ecatslab.com CSR under /simpleenroll should return HTTP 200
openssl req -new -newkey rsa:3072 -nodes \
  -keyout /tmp/good2.key -out /tmp/good2.csr \
  -subj "/CN=newdevice.ecatslab.com" \
  -addext "subjectAltName=DNS:newdevice.ecatslab.com"
openssl req -in /tmp/good2.csr -outform DER | base64 > /tmp/good2.b64
curl -sk --cacert /tmp/ca.pem -u est-user:<pw> \
  https://svs-rtp-ca2.cisco.com:8443/.well-known/est/simpleenroll \
  --data-binary @/tmp/good2.b64 \
  -H "Content-Type: application/pkcs10" \
  -H "Content-Transfer-Encoding: base64" \
  -o /tmp/good2.p7 -w "HTTP=%{http_code}\n" # expect 200

```

```

# BAD: a foreign domain should return HTTP 500 (subject validation error)
openssl req -new -newkey rsa:3072 -nodes \

```

```

-keyout /tmp/bad2.key -out /tmp/bad2.csr \
-subj "/CN=rogue.attacker.io"
openssl req -in /tmp/bad2.csr -outform DER | base64 > /tmp/bad2.b64
curl -sk --cacert /tmp/ca.pem -u est-user:<pw> \
https://svs-rtp-ca2.cisco.com:8443/.well-known/est/simpleenroll \
--data-binary @/tmp/bad2.b64 \
-H "Content-Type: application/pkcs10" \
-H "Content-Transfer-Encoding: base64" \
-o /tmp/bad2.p7 -w "HTTP=%{http_code}\n" # expect 500

```

3. Apache in front of the EST endpoint

Apache is configured via `~/est-lab/openxpki-config/contrib/apache2-openxpki-site.conf`. The file is mounted read-only into the OpenXPKI_WebUI container at `/etc/apache2/sites-enabled/openxpki.conf`. Changes on the host require `docker restart OpenXPKI_WebUI`.

Three pieces to get right.

3a. Disable global client-cert requests

By default the seed config sets `SSLVerifyClient optional_no_ca`, which makes Apache ask the client for a certificate on every TLS handshake. When IOS XE sees that request it presents whatever cert it happens to have, and OpenXPKI then tries to validate that cert as an on-behalf signer and rejects the enrolment.

Change the global setting to `none` and force TLS 1.2 so that Apache can later negotiate a client certificate on `/simpleenroll` via TLS renegotiation (which IOS XE's EST client supports; TLS 1.3 post-handshake auth is not negotiated by the client at the time of writing, AH10158):

```

<VirtualHost *:443>
    ServerAlias *
    DocumentRoot /var/www/

    # Force TLS 1.2 on the EST front end so that IOS XE can
    # authenticate with its existing client cert on /simpleenroll.
    SSLProtocol -all +TLSv1.2
    SSLHonorCipherOrder on

    SSLEngine On
    SSLCertificateFile      /etc/openxpki/tls/endentity/openxpki.crt
    SSLCertificateKeyFile   /etc/openxpki/tls/private/openxpki.pem

    SSLCACertificatePath    /etc/openxpki/tls/chain/
    SSLVerifyClient         none
    SSLVerifyDepth          3
    SSLOptions +StdEnvVars +ExportCertData

    <IfModule mod_headers.c>
        Use OxiForwardEnv SSL_CLIENT_S_DN
    
```

```
Use OxiForwardEnv SSL_CLIENT_CERT
</IfModule>
```

3b. Require HTTP Basic authentication on /simpleenroll

Anonymous /simpleenroll is how the seed config ships, which is fine for a first lab run but would be reckless in production. The router is bootstrapping from no cert and has to authenticate with *something*. RFC 7030 explicitly allows HTTP Basic for this case, and IOS XE 17.18+ implements it via enrollment http username ... password

Gate /simpleenroll on a credential file and scrub the Authorization header before the request is handed to OpenXPki (otherwise the backend would see the credential despite Apache already having consumed it):

```
<LocationMatch "^/\.well-known/est/(.+)/?simpleenroll$">
  AuthType Basic
  AuthName "EST Bootstrap"
  AuthUserFile /etc/openxpki/htpasswd/est.passwd
  Require valid-user
  # Strip the Authorization header before the request is proxied to
  # the OpenXPki backend so the enrollment workflow does not see it
  # (the auth decision has already been made by Apache).
  RequestHeader unset Authorization
</LocationMatch>
```

Per-device bootstrap credentials (recommended)

A single shared bootstrap password is a single blast-radius credential: if any device image or configuration management change leaks it, every router in the fleet has to be re-keyed at once. The production pattern is one entry per device, keyed on the device's own hostname:

```
pgiralt-c8kv-test:$2y$10$5fukG9m9LKEI2oNrx0sJaOK1lfNAU8nhHcLbnEmYCjNw1mAgPPrem
lab-c8kv-01:$2y$10$...another-random-hash...
lab-c8kv-02:$2y$10$...another-random-hash...
```

Each line is bcrypt (-B) at work-factor 10 (-C 10). Apache checks AuthUserFile on every request, so adding or removing a line takes effect immediately - no reload required.

Mint a new device credential on the CA host:

```
# Generate a 24-char random password and print it plus the bcrypt hash.
DEV_ID=pgiralt-c8kv-test
DEV_PW=$(openssl rand -base64 18 | tr -d '=/' | cut -c1-24)
htpasswd -B -C 10 -b ~/est-lab/openxpki-docker/openxpki-config/htpasswd/est.passwd "$DEV_ID"
"$DEV_PW"
echo "Push to device: $DEV_ID / $DEV_PW"
```

Push the password to the device's trustpoint (see chapter 6). The device's trustpoint username must match the htpasswd username exactly:

```
crypto pki profile enrollment EST-PROFILE
method-est
enrollment url https://svs-rtp-ca2.cisco.com
enrollment http username pgiralt-c8kv-test password 0 <pw-from-above>
```

IOS XE will re-encrypt the password at type 6 (AES) using the key `config-key password-encrypt master key`, which is why the *plaintext* is only exposed for the seconds it takes to paste the config.

If a device is decommissioned or its credential leaks, delete the line:

```
sed -i "/^pgiralt-c8kv-test:/d" \
~/est-lab/openxpki-docker/openxpki-config/htpasswd/est.passwd
```

That device can no longer onboard. Every other device is unaffected.

Rotation cadence (item 10)

A good baseline for production:

Event	Action
Routine	Rotate every 90 days
Staff change (PKI operator turnover)	Rotate all entries
Suspected leak	Rotate the affected entry immediately
Device retirement	Delete the line
One-shot bootstrap hardening	Delete the line once the device has a cert

The last row is worth calling out: once a device has enrolled, every subsequent renewal uses `/simpleenroll` and the device's own certificate (mTLS). The HTTP Basic credential is only needed again if the device is wiped and has to re-bootstrap. Some fleets choose to delete the bootstrap line as soon as the first certificate is issued, and re-mint it out-of-band only when a device is being re-deployed.

Two-hash rollover helper

A rotation has to avoid the window where the `htpasswd` file has already been updated to the new hash but the device still has the old password - during that window `/simpleenroll` would fail. The pattern is to keep **both** the old and new hashes valid across the rollover, under different usernames (e.g. `pgiralt-c8kv-test` and `pgiralt-c8kv-test-v2`). Apache matches on whichever username the client sends, so both are live simultaneously.

The repo ships `scripts/rotate-est-bootstrap-cred.sh` to automate this:

```
# Stage 1: add the new credential alongside the old one.
# Prints the new username and password to stdout.
./scripts/rotate-est-bootstrap-cred.sh prepare pgiralt-c8kv-test

# [Push the new username+password to the device out-of-band.]
# [Verify the device can still reach the CA and successfully re-enroll.]

# Stage 2: remove the old entry.
./scripts/rotate-est-bootstrap-cred.sh finalize pgiralt-c8kv-test
```

If the device fails to pick up the new credential, the old one is still live - no outage. The window is only truly closed at the `finalize` step.

Mount the credential file into the web container

The file has to be readable *inside* the Apache container. Mount it via a `docker-compose.override.yml` alongside the stock `openxpki-docker` compose file (example at `examples/openxpki/docker-compose.override.yml`):

```
services:
  web:
    volumes:
      - ./openxpki-config/htpasswd:/etc/openxpki/htpasswd:ro
```

After the override is in place, recreate the web container so the bind mount takes effect:

```
cd ~/est-lab/openxpki-docker
docker compose up -d web
```

3c. Security headers on the VirtualHost (item 11)

HSTS, X-Frame-Options, and X-XSS-Protection ship with the upstream vhost. Add four more standard hardening headers so the WebUI is covered end-to-end. The EST endpoints under `/.well-known/est/` speak binary PKCS#7 and are not affected by browser headers, so this is primarily about the admin UI surface:

```
# Set security headers.
#
# HSTS, X-Frame-Options and X-XSS-Protection were the upstream
# defaults. The additional headers below are layered on for the
# WebUI (the EST endpoints under /.well-known/est/ speak binary
# PKCS#7 and are not affected by browser headers, so this is
# primarily about hardening the admin UI surface).
#
# - X-Content-Type-Options: nosniff      Prevents MIME sniffing.
# - Referrer-Policy: no-referrer        Strips Referer on nav away.
# - Permissions-Policy                  Turns off browser features
#                                       the WebUI never needs
#                                       (camera, mic, geo, USB, etc.).
# - Content-Security-Policy              Locks script/style/image
#                                       loading to first-party. The
#                                       OpenXPki WebUI is a plain
#                                       Mojolicious app; no third-
#                                       party JS is expected.
<IfModule mod_headers.c>
Header set Strict-Transport-Security "max-age=31536000; includeSubDomains"
Header set X-Frame-Options           "DENY"
Header set X-XSS-Protection           "1; mode=block;"
Header set X-Content-Type-Options     "nosniff"
Header set Referrer-Policy            "no-referrer"
Header set Permissions-Policy         "accelerometer=(), camera=(), geolocation=(), gyros
cope=(), magnetometer=(), microphone=(), payment=(), usb=()"
Header set Content-Security-Policy    "default-src 'self'; script-src 'self'; style-src '
self' 'unsafe-inline'; img-src 'self' data:; font-src 'self' data:; connect-src 'self'; frame
-ancestors 'none'; base-uri 'self'; form-action 'self'; object-src 'none'"
</IfModule>
```

Verify from a client after docker restart OpenXPki_WebUI:

```
curl -skI https://svs-rtp-ca2.cisco.com:8443/ | \
  grep -iE 'strict-transport|x-frame|x-content-type|referrer-policy|permissions-policy|content-security-policy'
```

You should see all seven headers in the response.

3d. Re-enable client-cert verification only for /simpleenroll

Re-enrollment is the flow where the device already has a certificate, so mutual-TLS with that cert is both the natural and the strongest option. Scope the client-cert request to that one location:

```
# Require client cert for EST /simpleenroll only
<LocationMatch "/(.well-known/est/(.+)?)simpleenroll">
    SSLVerifyClient      optional_no_ca
    SSLVerifyDepth       3
    SSLOptions +StdEnvVars +ExportCertData +OptRenegotiate
</LocationMatch>

# EST (RFC7030)
RewriteRule "/(.well-known/est/(.+)?)$" \
    "unix:${OxiClientSocket}|http://localhost/$1" [P]
ProxyPassReverse /.well-known/est https://localhost/
```

+OptRenegotiate is what lets Apache upgrade an existing TLS session to request a client certificate. This is only possible with TLS 1.2, which is why section 3a forced SSLProtocol -all +TLSv1.2. TLS 1.3 would need post-handshake auth, which the IOS XE 17.18 EST client does not advertise (Apache's log records this as AH10158).

Full reference file: examples/openxpki/apache2-openxpki-site.conf.

3e. Apply the changes

```
# Reload the server workflow definitions (idempotent)
docker exec OpenXPki_Server openxpkitl reload server
docker exec OpenXPki_Client openxpkitl reload client

# Apache needs a restart for Location/Rewrite changes
docker restart OpenXPki_WebUI
```

4. Sanity checks

From the CA host, with the Root CA at /tmp/ca.pem:

```
# /cacerts is intentionally anonymous
curl -sk https://svs-rtp-ca2.cisco.com:8443/.well-known/est/cacerts \
    -o /tmp/cacerts.p7 -w "HTTP=%{http_code}\n"
# HTTP=200

# /simpleenroll must refuse anonymous requests
```

```

openssl req -new -newkey rsa:3072 -nodes \
    -keyout /tmp/t.key -out /tmp/t.csr \
    -subj "/CN=sanity.ecatslab.com" \
    -addext "subjectAltName=DNS:sanity.ecatslab.com"
openssl req -in /tmp/t.csr -outform DER | base64 > /tmp/t.b64

curl -sk --cacert /tmp/ca.pem \
    https://svs-rtp-ca2.cisco.com:8443/.well-known/est/simpleenroll \
    --data-binary @/tmp/t.b64 \
    -H "Content-Type: application/pkcs10" \
    -H "Content-Transfer-Encoding: base64" \
    -o /tmp/t.p7 -w "HTTP=%{http_code}\n"
# HTTP=401

# /simpleenroll succeeds when the Basic credential is supplied
curl -sk --cacert /tmp/ca.pem -u est-user:EstLabPass2026 \
    https://svs-rtp-ca2.cisco.com:8443/.well-known/est/simpleenroll \
    --data-binary @/tmp/t.b64 \
    -H "Content-Type: application/pkcs10" \
    -H "Content-Transfer-Encoding: base64" \
    -o /tmp/t.p7 -w "HTTP=%{http_code}\n"
# HTTP=200

```

If any of these return a different status, jump to Chapter 8.

5. What is still worth reviewing for production

Most of the production gaps the community usually hits are already closed by the configuration above: the `authorized_signer` rule is pinned to the `tls_dual` profile plus a subject regex, the profile enforces an FQDN allow-list, every issued cert carries CDP/SKI/AKI, the `htpasswd` file is per-device with a documented rotation cadence, and the WebUI vhost ships a full set of security headers. The following items are not addressed in this guide because they are deployment-dependent rather than pure configuration:

- Consider replacing HTTP Basic on `/simpleenroll` with a signed onboarding token (IEEE 802.1AR secure device identity, shared-secret one-time codes, etc.) if your device platform supports it. OpenXPki can validate such a token via the eligibility workflow, in which case you would also flip `allow_anon_enroll` to 0.
- Narrow `renewal_period` to a value that matches your certificate lifetime. The `010000` (365 days) used here is deliberately wide so that lab re-enrollments never get rejected for “too early”; in production with 1-year certs, `000060` (60 days) is a reasonable value.
- Put a WAF / rate-limiter (`mod_evasive`, `fail2ban` on the 401 log stream) in front of `/simpleenroll` to blunt credential-stuffing attempts against the `htpasswd` file.
- Move the Issuing CA signing key out of the software `svault` and into an HSM/KMS through OpenXPki’s PKCS#11 token support.

Next: Chapter 6.

Chapter 6. IOS XE EST Client Configuration

This chapter is the router-side companion to Chapter 5. At the end of it, the IOS XE device will have:

- An RSA key pair dedicated to EST (EST-CLIENT-KEY).
- A trustpoint that holds the CA chain (EST-ROOT).
- An enrollment profile using EST (EST-PROFILE).
- A client trustpoint that holds the router's own device certificate (EST-CLIENT), enrolled against the enrollment profile.

1. Prerequisites on the router

```
router01#show version | include IOS XE Software, Version
Cisco IOS XE Software, Version 17.18.2

router01#show clock detail
20:53:26.427 UTC Wed Apr 22 2026
Time source is NTP

router01#show ip interface brief | include up
GigabitEthernet1      192.0.2.50   YES DHCP    up           up

router01#ping ca.example.com
!!!!!

router01#show ip http client
HTTP client current connections:
... (service is enabled)
```

If ip http client is disabled on your platform, enable it globally:

```
configure terminal
ip http client source-interface GigabitEthernet1
end
```

2. Generate a dedicated RSA key pair

Do not reuse an existing key (especially one used for SSH). The key is the binding identity of the certificate and must be rotatable independently.

This guide standardizes on a 3072-bit general-purpose RSA key. 3072 bits is the current NIST recommendation (FIPS 186-5 / SP 800-57), equivalent to roughly 128-bit symmetric strength, and is what you should use for any new deployment. 2048 bits is still acceptable but is slated to fall out of compliance in the second half of this decade; only use it when a platform cannot keep up with a 3072 handshake. IOS XE also supports ECDSA on P-256 or P-384 curves (eckeypair / crypto key generate ec keysizes), which is a

stronger alternative if your CA, the rest of your trust chain, and the relying parties can all consume EC certificates.

```
configure terminal
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
end
```

Verify:

```
router01#show crypto key mypubkey rsa EST-CLIENT-KEY
Key name: EST-CLIENT-KEY
Usage: General Purpose Key
Key is not exportable. Redundancy enabled.
Key Data:
 30820122 300D0609 2A864886 F70D0101 ...
```

If you ever hit the “transaction ID” stuck-workflow problem described in Chapter 8, regenerate the key:

```
configure terminal
crypto key zeroize rsa EST-CLIENT-KEY
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
end
```

3. Authenticate the Root CA

This is the step that installs the OpenXPki Root into the router’s trust store so that TLS and enrollment can validate the chain.

Create a trustpoint that holds the Root:

```
configure terminal
crypto pki trustpoint EST-ROOT
enrollment terminal
revocation-check none
exit
end
```

Then paste the Root PEM:

```
router01#configure terminal
router01(config)#crypto pki authenticate EST-ROOT
Enter the base 64 encoded CA certificate.
End with a blank line or the word "quit" on a line by itself

-----BEGIN CERTIFICATE-----
MIIB.... (Root CA from ~/est-lab/root-ca.pem) ....
-----END CERTIFICATE-----
quit
Certificate has the following attributes:
  Fingerprint MD5: ...
  Fingerprint SHA1: ...
```

```
% Do you accept this certificate? [yes/no]: yes
Trustpoint CA certificate accepted.
```

Verify:

```
router01#show crypto pki certificates EST-ROOT
CA Certificate
  Status: Available
  Certificate Serial Number (hex): 258EAB89...
  Issuer:  cn=OpenXPKI Root DUMMY CA 20260422 ...
  Subject: cn=OpenXPKI Root DUMMY CA 20260422 ...
```

4. Configure the EST enrollment profile

The profile tells the router where the EST server lives, that EST is the method to use, *and* how to authenticate to the server for each of the two EST operations:

```
configure terminal
crypto pki profile enrollment EST-PROFILE
  method-est
  enrollment url https://ca.example.com:8443/.well-known/est/
  enrollment credential EST-CLIENT
  enrollment http username est-user password 0 EstLabExamplePw!
exit
end
```

Critical details: - `method-est` must appear before `enrollment url`. Without it, IOS XE tries to interpret the URL as a SCEP endpoint and the authenticate step fails with “failed to open file”. - The URL must end with a trailing slash (`/ .well-known/est/`). IOS XE appends operation names such as `cacerts` and `simpleenroll` directly. - The hostname in the URL must match the CN/SAN of the EST server TLS certificate. No IP address shortcuts. - `enrollment credential EST-CLIENT` points at the trustpoint that will hold the device cert once enrollment completes. IOS XE uses that cert as the TLS client certificate when it calls `/simplereenroll`. See section 8 for why this is required. - `enrollment http username ... password ...` is the HTTP Basic credential the router presents on `/simpleenroll` during the initial bootstrap (when it has no cert yet). Enter the password as `password 0 <plaintext>; if password encryption aes + key config-key password-encrypt` is configured (see 4b), IOS XE will re-encode it as type 6 immediately.

4a. Why both credentials are needed

This is the production-grade shape of the configuration. The two directives serve different EST operations:

EST operation	Used by	Router authenticates with	Apache enforces
<code>/cacerts</code>	Initial + renewal	Nothing (anonymous)	Nothing
<code>/simpleenroll</code>	Initial bootstrap	HTTP Basic (<code>enrollment http ...</code>)	<code>AuthType Basic + httpasswd</code>
<code>/simplereenroll</code>	Every renewal	mTLS with current device cert	<code>SSLVerifyClient optional_no_ca</code>

At bootstrap time the device has no certificate, so mTLS is not an option - Basic is the only practical way to say “yes, this really is a Cisco router, not some unknown client”. Once the first certificate is installed, every

renewal uses that cert for mTLS, which is strictly stronger than Basic: the credential never crosses the wire in cleartext and it rotates automatically on every regenerate cycle. The Basic credential is only consulted again if the device ever has to bootstrap from scratch (zeroized key, wiped config, etc.).

IOS XE 17.18 is per-operation smart about the two credentials: it attaches the `Authorization: Basic ...` header only to `/simpleenroll` requests and omits it on `/simplereenroll` (confirmed on the lab by logging `%{Authorization}i` in the Apache access log for one bootstrap cycle and one renewal cycle). That means:

- Leaving both directives in the enrollment profile after the first enrollment is functionally safe - the Basic credential is never transmitted during renewals and therefore cannot be replayed from a wiretap of a routine re-enrollment.
- The useful lifetime of the stored credential is “until you ever have to re-bootstrap this device”. If that ever becomes never (e.g. an operational model where a wiped device is re-provisioned out-of-band with a fresh credential pushed at the same time), you can delete the enrollment `http` line from the profile after the first successful enrollment. The cost is that you must treat “cert loss” as a manual event.
- The credential still lives on the device as a long-term secret, so standard shared-secret hygiene applies: type-6 at rest (see 4b), per device or per small group if your PKI tooling can mint one-off `htpasswd` entries, and rotation on the same cadence as any other infrastructure password.

Because the credential is only touched on `/simpleenroll`, **rotating the htpasswd on the CA does not break devices that already have a valid certificate**. Their renewals (and ad-hoc manual `crypto pki enroll` calls) hit `/simplereenroll` and authenticate via mTLS, which does not read the `htpasswd` file at all. The stale credential becomes relevant only if a device has to bootstrap again from scratch - which includes the non-obvious case of a certificate that expired before the device could renew (e.g. extended outage spanning the full renewal window). For fleets where that failure mode matters, keep at least two valid `htpasswd` entries during a rotation:

```
est-user:$2y$05$<old-bcrypt-hash>
est-user:$2y$05$<new-bcrypt-hash>
```

Apache accepts either, so devices configured with the old credential stay bootstrap-capable until every router has been re-configured with the new one. Drop the old entry after the rollover window closes.

If you can afford the blast-radius improvement, mint per-device or per-group `htpasswd` entries (e.g. `est-<serial>`) so a rotation affects exactly one device instead of the whole fleet.

4b. Protect the bootstrap password at rest

The enrollment `http username ... password ...` line stores a long-lived credential in the router’s configuration. Leaving it in type-0 (plaintext) form in running-config / startup-config is a compliance red flag. Turn on AES-encrypted passwords so that IOS XE stores it as type 6:

```
configure terminal
key config-key password-encrypt <master-key-passphrase>
password encryption aes
end
```

key config-key installs a device-specific master key that encrypts every type-6 password at rest. After this is set, re-enter the enrollment credential so IOS XE re-encodes it:

```
configure terminal
crypto pki profile enrollment EST-PROFILE
  no enrollment http username est-user password 0 EstLabExamplePw!
  enrollment http username est-user password 0 EstLabExamplePw!
exit
end
```

A show running-config | section EST-PROFILE will now show the password as password 6 <opaque>, for example:

```
crypto pki profile enrollment EST-PROFILE
method-est
enrollment url https://ca.example.com:8443/.well-known/est/
enrollment credential EST-CLIENT
enrollment http username est-user password 6 <opaque-type6-string>
```

Two operational notes about the master key:

- The master key never leaves the device. If you ever restore this config onto a different router (DR, RMA), the new device cannot decrypt type-6 strings until you install the *same* master key with key config-key password-encrypt. Record the passphrase out-of-band (e.g. in a password manager) at the time you provision the device.
- Rotating the master key (key config-key password-encrypt <new> decrypt <old>) is supported but rewrites every type-6 entry on the device. Schedule a maintenance window.

5. Configure the device (client) trustpoint

This is the trustpoint that will hold the issued device certificate:

```
configure terminal
crypto pki trustpoint EST-CLIENT
  enrollment profile EST-PROFILE
  subject-name CN=router01.example.com
  fqdn router01.example.com
  rsakeypair EST-CLIENT-KEY 3072
  revocation-check crl none
  hash sha256
  auto-enroll 80 regenerate
exit
end
```

Field-by-field:

Field	Why it matters
enrollment profile	Delegates enrollment to EST-PROFILE. Do not also set enrollment url on this trustpoint.

Field	Why it matters
<code>subject-name</code>	Exact DN to place in the CSR. OpenXPki profiles can transform this, but IOS XE sends what you configure.
<code>fqdn</code>	Drives the SAN DNS entry. Must match the real device hostname that your application will use.
<code>rsa-keypair EST-CLIENT-KEY 3072</code>	Binds the trustpoint to the dedicated EST key and records the bit size so that timer-driven regeneration re-creates it at 3072 bits. Supply a single bit count (general-keys); two counts like <code>3072 3072</code> switch the trustpoint into usage-keys mode and do not match a general-purpose key.
<code>revocation-check crl none</code>	Pull the published CRL from the CDP that OpenXPki stamps into every issued certificate, and fail-open to no-check if the CDP is unreachable. CRLs are refreshed on validity expiry; use <code>clear crypto pki crl EST-CLIENT</code> to force an immediate refresh. <code>crl</code> alone (no <code>none</code>) would fail-closed and block all TLS sessions if the CDP ever went down.
<code>hash sha256</code>	Signature hash for the CSR. SHA-1 is deprecated and IOS XE refuses it on 17.18+.
<code>auto-enroll 80 regenerate</code>	Timer-driven renewal at 80% of the cert's lifetime with a brand-new key. See the call-out box below.

The regenerate keyword is a best practice

Put `regenerate` on the `auto-enroll` line in every production trustpoint.

Without `regenerate`, IOS XE will submit every re-enrollment CSR with the **same** RSA public key the device has used since the first enrollment. That creates two problems:

1. The certificate's asymmetric key never rotates. A compromise of that key compromises every cert ever issued for that trustpoint, forever. `regenerate` caps the blast radius at a single cert lifetime.
2. OpenXPki (and several other EST servers) deduplicate `/simplereenroll` responses on `SubjectPublicKeyInfo`. When the server sees the same public key as the cert the client is presenting, it returns the **existing** certificate rather than issuing a new one. The effect is that the device never actually gets a new certificate - it just keeps reminding the old one until it expires. `regenerate` forces a fresh key and therefore a fresh cert every cycle.

Recommended minimum for any EST-based trustpoint:

```
crypto pki trustpoint EST-CLIENT
auto-enroll 80 regenerate
```

The `80` can be tuned - 50% is useful in labs, 75-85% is typical in production. The `regenerate` keyword itself should always be present.

6. Authenticate the EST server (pull CA chain via EST)

```
configure terminal
crypto pki authenticate EST-CLIENT
end
```

On success:

```
Certificate has the following attributes:
  Fingerprint MD5: 1197C04C ...
  Fingerprint SHA1: 9C384AEF ...
% Do you accept this certificate? [yes/no]: yes
Trustpoint CA certificate accepted.
```

If this step fails with % Error in receiving Certificate Authority certificate, check: - That method-est was configured on EST-PROFILE. - That the router can resolve the FQDN and reach TCP/8443. - That the server TLS certificate has the FQDN in both CN and SAN.

7. Enroll for the device certificate

```
router01#configure terminal
router01(config)#crypto pki enroll EST-CLIENT
% Start certificate enrollment ..

% The subject name in the certificate will include: CN=router01.example.com
% The subject name in the certificate will include: router01.example.com
% Include the router serial number in the subject name? [yes/no]: no
% Include an IP address in the subject name? [no]: no
Using EST to request certificate from CA? [yes/no]: yes
% Certificate request sent to Certificate Authority
% The 'show crypto pki certificate verbose EST-CLIENT' command will show the fingerprint.
```

The router will poll the EST server asynchronously. If OpenXPki is configured for auto-approval (as this guide sets up), the certificate arrives within a few seconds. Verify:

```
router01#show crypto pki certificates verbose EST-CLIENT
Certificate
  Status: Available
  Certificate Serial Number (hex): 04FF494AA868731AD140
  Issuer: cn=OpenXPki Issuing DUMMY CA 20260422
  Subject: cn=router01.example.com
  Validity Date: 20:53:28 UTC Apr 22 2026 -> 20:53:28 UTC Apr 22 2027
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public Key: (3072 bit)
  Associated Trustpoints: EST-CLIENT
CA Certificate
...
```

8. Re-enrollment and renewal

With the device certificate in place, `/simplereenroll` is the endpoint the router uses for all subsequent renewals. IOS XE automatically authenticates with its current device certificate (via mTLS) rather than the Basic credential that was used for the bootstrap. The bootstrap credential is only consulted again if the device ever has to re-enroll from scratch (for example after a zeroize).

Two things on the EST-PROFILE that make this work were already configured back in section 4:

- `enrollment credential EST-CLIENT` - points Apache's `+OptRenegotiate` at the trustpoint that holds the current device cert. Without this, the router has no client cert to present on `/simplereenroll` and Apache logs `self_auth failed`.
- `enrollment http username ... password ...` - the Basic credential for bootstrap. IOS XE attaches this only to `/simpleenroll` requests; on `/simplereenroll` the header is omitted entirely (see the per-operation discussion in section 4a). Leaving the directive in place at renewal time is harmless, and useful if you ever need to bootstrap the device from scratch again.

The automatic renewal configuration was already added in section 5 (`auto-enroll 80 regenerate`). Re-read the “The regenerate keyword is a best practice” box above if you are tempted to drop the regenerate keyword - you almost never want to.

Quick summary of what that line does at the 80% mark of the cert's lifetime:

1. IOS XE generates a fresh EST-CLIENT-KEY RSA pair, sized according to the `rsa keypair ... 3072` directive on the trustpoint.
2. It builds a new CSR with that fresh public key and the same subject / SAN as the active cert.
3. It opens a TLS 1.2 session to the EST server, authenticates with the current device cert via mTLS, and posts the CSR to `/simplereenroll`.
4. The new cert is installed and the previous one is retired.

For manual, on-demand renewals (CI/CD, ad-hoc tests, compromise response) see Chapter 7 section 7.2.

9. Using the issued certificate

You can now bind EST-CLIENT to any service that consumes PKI trustpoints, for example:

- IKEv2 (`crypto ikev2 profile ... pki trustpoint EST-CLIENT`)
- IOS HTTPS server (`ip http secure-trustpoint EST-CLIENT`)
- SSH server host-key identity (`ip ssh server trustpoint sign EST-CLIENT`)

Next: Chapter 7 for the full validation battery.

Chapter 7. Testing and Verification

This chapter is a checklist of the “does it work?” commands, grouped by layer, so that if something is broken you can quickly narrow it down before diving into Chapter 8.

1. OpenXPKI stack health

```
docker compose -f ~/est-lab/openxpki-docker/docker-compose.yml ps
```

All four containers should be healthy. OpenXPKI_WebUI occasionally reports unhealthy during its first 10 seconds of startup; recheck.

```
curl -sk https://svs-rtp-ca2.cisco.com:8443/healthcheck/ping
```

Should print OK or a small JSON status object. A non-zero exit code (connection refused) means Apache is not up; run `docker logs OpenXPKI_WebUI`.

2. EST endpoints via curl

From the CA host or any Linux client that trusts the Root:

```
# cacerts always works without auth
curl -sk https://svs-rtp-ca2.cisco.com:8443/.well-known/est/cacerts \
  -o /tmp/cacerts.p7 -w "HTTP=%{http_code}\n"
# HTTP=200
```

```
# csrattrs is optional, returns 204 if nothing configured
curl -sk https://svs-rtp-ca2.cisco.com:8443/.well-known/est/csrattrs \
  -o /tmp/csrattrs -w "HTTP=%{http_code}\n"
```

Build a throwaway CSR and enroll. Because /simpleenroll is protected by HTTP Basic auth, you must supply the same credential that the router uses (enrollment http username ... password ...):

```
openssl req -new -newkey rsa:3072 -nodes \
  -keyout /tmp/t.key -out /tmp/t.csr \
  -subj "/CN=sanity.ecatslab.com" \
  -addext "subjectAltName=DNS:sanity.ecatslab.com"
```

```
openssl req -in /tmp/t.csr -outform DER | base64 > /tmp/t.b64
```

```
# Anonymous request is refused at the edge
curl -sk --cacert /tmp/ca.pem \
  https://svs-rtp-ca2.cisco.com:8443/.well-known/est/simpleenroll \
  --data-binary @/tmp/t.b64 \
  -H "Content-Type: application/pkcs10" \
  -H "Content-Transfer-Encoding: base64" \
  -o /dev/null -w "HTTP=%{http_code}\n"
# HTTP=401
```

```
# Authenticated request succeeds
curl -sk --cacert /tmp/ca.pem -u est-user:EstLabPass2026 \
  https://svs-rtp-ca2.cisco.com:8443/.well-known/est/simpleenroll \
  --data-binary @/tmp/t.b64 \
  -H "Content-Type: application/pkcs10" \
  -H "Content-Transfer-Encoding: base64" \
  -o /tmp/t.p7 -w "HTTP=%{http_code}\n"
# HTTP=200
```

Decode and inspect the issued cert:

```
openssl base64 -d -A -in /tmp/t.p7 | \
  openssl pkcs7 -inform DER -print_certs | \
  openssl x509 -noout -subject -issuer \
    -ext subjectAltName -ext extendedKeyUsage
```

Expected:

```
subject=CN = sanity.ecatslab.com
issuer=CN = OpenXPKI Issuing DUMMY CA 20260422
X509v3 Subject Alternative Name:
  DNS:sanity.ecatslab.com
X509v3 Extended Key Usage:
  TLS Web Client Authentication, TLS Web Server Authentication
```

3. TLS server verification (hostname + chain)

```
openssl s_client -connect svb-rtp-ca2.cisco.com:8443 \
  -servername svb-rtp-ca2.cisco.com \
  -showcerts -verify_return_error < /dev/null 2>&1 | \
  grep -E 'Verify return code|subject=|issuer=|DNS:' | head -10
```

Expected:

```
0 s:CN = svb-rtp-ca2.cisco.com
  i:CN = OpenXPKI Issuing DUMMY CA 20260422
1 s:CN = OpenXPKI Issuing DUMMY CA 20260422
  i:CN = OpenXPKI Root DUMMY CA 20260422
Verify return code: 0 (ok)
```

If you see Verify return code: 19 (self-signed certificate in certificate chain), the Root is not in your local trust store. Trust ~/est-lab/root-ca.pem on your workstation or pass -CAfile.

4. IOS XE state checks

```
pgiralt-c8kv-test#show crypto pki trustpoints status
pgiralt-c8kv-test#show crypto pki profile

pgiralt-c8kv-test#show crypto pki certificates EST-CLIENT
Certificate
```

```
Status: Available
Certificate Serial Number (hex): 04FF494AA868731AD140
Issuer: cn=OpenXPki Issuing DUMMY CA 20260422
Subject: cn=pgiralt-c8kv-test.ecatslab.com
Validity Date: 20:53:28 UTC Apr 22 2026 -> 20:53:28 UTC Apr 22 2027
```

Verbose view with extensions:

```
pgiralt-c8kv-test#show crypto pki certificates verbose EST-CLIENT
```

Look specifically for:

```
X509v3 Key Usage: A0000000
    Digital Signature
    Key Encipherment
X509v3 Subject Alternative Name:
    pgiralt-c8kv-test.ecatslab.com
Extended Key Usage:
    Server Auth
    Client Auth
```

Renewal timers:

```
pgiralt-c8kv-test#show crypto pki timers
```

5. End-to-end transaction trace

When a request misbehaves, capture the full life of the transaction:

- On the router:

```
debug crypto pki transactions
debug crypto pki messages
debug ssl openssl errors
```

- On the CA host, tail both the client and workflow logs:

```
docker exec OpenXPki_Client tail -f /var/log/openxpki-client/est.log
docker exec OpenXPki_Server tail -f /var/log/openxpki-server/workflows.log
docker exec OpenXPki_Server tail -f /var/log/openxpki-server/catchall.log
```

Typical successful trace on the server side:

workflows.log:

```
1791 Rendering subject: CN=pgiralt-c8kv-test.ecatslab.com,DC=Test Deployment,DC=OpenXPki,DC=org
1791 Eligibility check for est.default.eligible.initial granted
1791 Approval points for workflow #1791: 1
1791 persisted csr for CN=pgiralt-c8kv-test.ecatslab.com,DC=Test Deployment,DC=OpenXPki,DC=org with csr_serial 1279
1791 start cert issue for serial 1279, workflow 1791
1791 Certificate CN=pgiralt-c8kv-test.ecatslab.com,... issued by ca-signer-1
```

If the log skips any of these steps, identify which condition failed and consult Chapter 8.

6. Revocation check

The router trustpoint is configured with `revocation-check crl none`, so it pulls the CRL from the CDP stamped into each issued certificate. You can verify CRL issuance and availability independently on the CA side with:

```
docker exec OpenXPKI_Server oxi api search_crl --realm democa
```

And list active certificates for a given subject:

```
docker exec OpenXPKI_Server oxi api search_cert \  
  --realm democa --subject "CN=pgiralt-c8kv-test.ecatslab.com%"
```

7. Re-enrollment testing

Once the device has a valid certificate, the next thing to validate is the renewal path: the router reaching `/simplereenroll` with mutual TLS and receiving a new certificate without any operator interaction. Two complementary tests cover this end to end - a manual, on-demand renewal and a timer-driven automatic renewal against a certificate with a very short validity.

7.1 How OpenXPKI decides what to return

OpenXPKI enforces two rules that shape re-enrollment behavior and that are often surprising the first time you hit them.

- **Public-key deduplication.** If the re-enrollment CSR carries the **same** `SubjectPublicKeyInfo` as the certificate the device is presenting in its TLS client auth, OpenXPKI will return the existing certificate rather than issuing a new one. This is by design (it prevents an attacker or a misbehaving client from burning through cert slots with replayed CSRs), but it means that a pure “renew with the same key” test looks like a no-op: the serial number on the router does not change.

To force a genuinely new certificate the router must present a CSR with a fresh key pair. On IOS XE the cleanest way is to put the trustpoint on `auto-enroll N regenerate`, which regenerates the RSA key pair right before each automatic renewal. For a manual test, `zeroize` and `regenerate` the key on demand (section 7.2 below).

- **renewal_period window.** In `realm/democa/est/default.yaml` the `renewal_period` option sets how far from the certificate’s `notAfter` OpenXPKI is willing to accept a renewal request. The default “000060” (60 days) is tuned for production year-long certs. In the lab, if you issue a 1-year cert and try to renew it 10 minutes later, OpenXPKI rejects the request because you are more than 60 days away from expiry.

For a lab where you renew early and often, loosen this to the full certificate lifetime or more:

```
# Allow re-enrollment at any point in the certificate's validity.  
renewal_period: "010000"
```

Reload the server and client after editing:

```
docker exec OpenXPKI_Server openxpkictl reload server
docker exec OpenXPKI_Client openxpkictl reload client
```

7.2 Manual re-enrollment on demand

This exercise proves that the mutual-TLS re-enrollment path works. It does not require any change in validity period.

Prerequisites: the EST-PROFILE enrollment profile on the router must carry an enrollment credential reference to the currently active trustpoint so that IOS XE can present the device cert during the TLS handshake. Chapter 6 section 4 already configures this as part of the normal production profile, alongside the HTTP Basic credential that the router uses on /simpleenroll:

```
configure terminal
crypto pki profile enrollment EST-PROFILE
method-est
enrollment url https://svs-rtp-ca2.cisco.com:8443/.well-known/est/
enrollment credential EST-CLIENT
enrollment http username est-user password 6 <type6-string>
end
```

To force a new certificate (different serial number), regenerate the key before enrolling:

```
configure terminal
crypto key zeroize rsa EST-CLIENT-KEY
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
crypto pki enroll EST-CLIENT
end
```

Expected result:

```
pgiralt-c8kv-test#show crypto pki certificates EST-CLIENT | include Serial|date
Certificate Serial Number (hex): 0AFF14EA02131EBFCF33
start date: 00:10:34 UTC Apr 23 2026
renew date: 00:17:26 UTC Apr 23 2026
```

If you skip the key regeneration step and just run `crypto pki enroll EST-CLIENT`, the router still calls /simpleenroll successfully but the returned certificate is the one already on disk (same serial, same dates). That is the dedup rule from 7.1 in action, not a bug.

Server-side confirmation. Tail the EST client log on the CA while running the enroll command:

```
docker exec OpenXPKI_Client tail -f /var/log/openxpki-client/est.log
```

You should see an Authenticated client DN line carrying the router's subject, followed by a new workflow ID and a Certificate ... issued event in workflows.log.

7.3 Automatic renewal with a short-lived certificate

The scripted way to exercise the renewal timer without waiting months.

Step 1 - Temporarily shorten the cert lifetime in realm/democa/profile/tls_dual.yaml:


```
# 15-minute validity; revert after the test.
```

```
validity:
  notafter: "+0000000015"
```

OpenXPki uses a positional duration string (YYYYMMDDhhmmss), so +0000000015 means “now + 15 minutes”.

Step 2 - Restart the server and client containers so the profile change takes effect. A plain reload is not reliably enough for profile modifications:

```
docker compose -f ~/est-lab/openxpki-docker/docker-compose.yml \
  restart server client webui
```

Step 3 - Configure the router’s trustpoint to regenerate the key at renewal time. 50 percent is a convenient threshold when the certificate lives for only 15 minutes:

```
configure terminal
crypto pki trustpoint EST-CLIENT
  auto-enroll 50 regenerate
end
```

Step 4 - Ensure the cert chain and key are fresh, then enroll:

```
configure terminal
no crypto pki certificate chain EST-CLIENT
crypto key zeroize rsa EST-CLIENT-KEY
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
crypto pki authenticate EST-CLIENT
crypto pki enroll EST-CLIENT
end
```

Note: no crypto pki certificate chain EST-CLIENT removes both the device cert *and* the Issuing CA cert from that trustpoint, which is why the authenticate step has to run again before the enroll.

Step 5 - Watch the renewal cycles. Expected behavior on the router:

- At t = 0 the device receives a 15-minute certificate.
- At t = 7.5 minutes (50 percent of lifetime) the auto-renewal timer fires. The router regenerates EST-CLIENT-KEY, posts a new CSR to /simplereenroll, and installs the replacement certificate (different serial, new start/end dates).
- The timer then fires again 7.5 minutes into the next cert, and so on.

You can observe the cadence with:

```
pgiralt-c8kv-test#show logging | include CERT_RENEW|CERT_ENROLL|KEY_ADD|KEY_DELETE
```

and confirm each new serial with:

```
pgiralt-c8kv-test#show crypto pki certificates EST-CLIENT | include Serial|date
```

Sample excerpt from a successful loop against a 3072-bit trustpoint:

```
00:45:39: Certificate 10FF2A23FDE66CD4C15B installed, RSA Public Key (3072 bit)
```

```
00:51:49: %PKI-6-CERT_RENEW_AUTO: Auto renewing router certificate of trustpoint EST-CLIENT
```

```
00:51:49: %CRYPTO_ENGINE-5-KEY_ADDITION: A key named EST-CLIENT-KEY# has been generated
00:51:51: %CRYPTO_ENGINE-5-KEY_DELETED: A key named EST-CLIENT-KEY has been removed
00:51:52: Certificate 11FFD51910DB98E53515 installed, RSA Public Key (3072 bit)
```

Each renewal produces a new serial **and** a new public key because of the regenerate keyword; without it the router would keep presenting the same key and OpenXPki would dedup the response back to the previous cert.

Step 6 - When the test is complete, revert both sides:

- Remove the validity: override from `tls_dual.yaml` so newly issued certs go back to the default 1-year validity.
- Dial `auto-enroll` back to a realistic threshold, typically `auto-enroll 80 regenerate` for year-long certs.
- Restart the stack (`docker compose restart server client webui`) after editing `tls_dual.yaml`.

7.4 What to do if the 50-percent renewal turns into an initial enroll

In the first cycle after removing the cert chain, IOS XE may log a `CERT_ENROLL_AUTO: Auto initial enrollment` message (instead of `CERT_RENEW_AUTO`) because the old cert has expired before the renewal timer had a chance to fire. That is fine - the device simply falls back to `/simpleenroll` and re-bootstraps. The very next cycle, with a fresh in-validity cert on disk and an enrollment credential configured, uses `/simplereenroll` as expected.

8. Clean-up after a test burst

If you enroll dozens of certificates during a debugging session, retire the old ones so the active cert count does not grow:

```
docker exec OpenXPki_Server oxi api revoke_cert \
  --realm democa --identifier <cert-identifier> \
  --reason superseded
```

Or simply wait for them to expire - 1 year by default.

Chapter 8. Troubleshooting and Common Issues

Every failure listed here was hit during the reference lab build. The items are ordered roughly from the most common to the most obscure. When a section mentions a log file path, it is inside the matching container (`docker exec OpenXPKI_Server ...` or `docker exec OpenXPKI_Client ...`).

- Stack start fails
- `curl` to `/cacerts` returns 403 or 500
- `simpleenroll` returns 401 Unauthorized
- EST request returns 202 Pending
- `oxi workflow` commands fail with `ACCESS_NOT_ALLOWED_FOR_USER`
- Issued cert missing SAN or Client ECU
- `crypto pki authenticate EST-CLIENT` fails
- Enroll keeps failing with `SIGNER_NOT_AUTHORIZED`
- IOS XE “Include an IP address” prompt freezes automation
- Clock skew errors
- Hostname mismatch during TLS handshake
- Reenrollment fails with TLS handshake failure
- Reenrollment returns the same certificate
- Reenrollment rejected as outside the renewal window
- Auto-regenerate fails with “Querying key pair failed”
- Can't change RSA key type here
- OpenXPKI refuses to parse the CSR

Stack start fails

Symptoms: - `docker compose up -d` reports services are unhealthy. - OpenXPKI_Server restarts in a loop.

Causes and fixes: - `##SVAULTKEY##` placeholder is still in `crypto.yaml`. Replace it with a 64-hex-character value (see Chapter 2). - Database was not ready when the server started. Wait 30 seconds and run `docker compose up -d` again. The compose file's `depends_on` only checks for container start, not healthiness. - You are running Docker inside WSL and the bind mounts do not expose proper Linux permissions. Move the checkout to a native Linux filesystem.

curl to /cacerts returns 403 or 500

Symptoms: - `curl -sk https://svs-rtp-ca2.cisco.com:8443/.well-known/est/cacerts` returns a non-200 code or an HTML error page.

Causes and fixes: - The OpenXPKI_Client container is stopped. Happens if `openxpkictl reload client` is issued without a follow-up `start`. Run `docker compose -f ~/est-lab/openxpki-docker/docker-compose.yml start client`. - The `realm/democa/est/default.yaml` file is missing. The EST endpoint name (default) is appended to the URL path; the file under `est/` must match. - The Apache `vhost` was edited but the web container was not restarted. Run `docker restart OpenXPKI_WebUI`.

simpleenroll returns 401 Unauthorized

Symptoms: - `curl` (or the router) receives HTTP 401 on `/.well-known/est/simpleenroll`. - Apache `est_access.log` shows a line with no authenticated user and `"POST /.well-known/est//simpleenroll HTTP/1.1" 401`. - On the router, `debug crypto pki transactions` reports Unauthorized and the enrollment never completes.

Causes and fixes: - The router is not sending a credential. Confirm that the enrollment profile includes an `enrollment http username ... password ... directive`. `enrollment url https://user:pass@host/...` is explicitly ignored by IOS XE; only the dedicated directive works. - The supplied Basic credential does not match the `htpasswd` file. Regenerate it with `htpasswd -Bbn est-user '<password>'` and make sure the container is reading the updated file (`docker exec OpenXPKI_WebUI cat /etc/openxpki/htpasswd/est.passwd`). - The `docker-compose.override.yml` bind mount is missing or mounted in the wrong path. The file must surface at `/etc/openxpki/htpasswd/est.passwd` inside the web container, exactly where `AuthUserFile` in the Apache `vhost` points. Run `docker compose up -d web` after fixing the override. - After a successful authenticated request you will still see *two* log entries: a 401 followed a millisecond later by a 200 by `est-user`. That is normal Basic-auth challenge-response behavior, not a failure.

EST request returns 202 Pending

Symptoms: - `curl -w "%{http_code}"` returns 202 and the body is a small JSON or empty response. - Log shows `Eligibility check for est.default.eligible.initial failed`.

Cause: even with `approval_points: 0`, OpenXPKI does not auto-approve if the request fails eligibility. By default initial enrollment eligibility is 0 (manual review required).

Fix: set both values in `est/default.yaml`:

```
policy:
  approval_points: 0

eligible:
  initial:
    value: 1
```

Reload the server:

```
docker exec OpenXPKI_Server openxpkictl reload server
docker exec OpenXPKI_Client openxpkictl reload client
```

oxi workflow commands fail with ACCESS_NOT_ALLOWED_FOR_USER

Symptom: - oxi workflow show --realm democa --id 511 returns
I18N_OPENXPki_UI_WORKFLOW_ACCESS_NOT_ALLOWED_FOR_USER.

Cause: the CLI admin user is configured with role System, which does not have creator: any permissions over workflows started by anonymous users.

Fix: in openxpki-config/config.d/system/cli.yaml, change:

```
auth:
  admin:
    role: RA Operator
```

and reload the server.

Issued cert missing SAN or Client EKU

Symptom: - Issued device cert has only serverAuth EKU and no SAN. - IOS XE cannot use the certificate for mutual TLS on /simplereenroll, and any other outbound mTLS feature on the router (TACACS+ over TLS, RADIUS/TLS / RadSec, NETCONF over TLS with client-cert auth, etc.) will fail the handshake or fall back to password auth.

Cause: the EST endpoint profile is pointing at tls_server or a custom profile with subject_alt_name.copy: 0.

Fix: in realm/democa/est/default.yaml, set:

```
profile:
  cert_profile: tls_dual
```

and ensure realm/democa/profile/tls_dual.yaml has:

```
extensions:
  extended_key_usage:
    server_auth: 1
    client_auth: 1
  subject_alt_name:
    copy: 1
```

Restart both server and client containers (a reload is not always enough for profile changes):

```
docker compose restart server client
```

crypto pki authenticate EST-CLIENT fails

Symptom:

```
% Error in receiving Certificate Authority certificate: status = FAIL
% Error: failed to open file.
```

Causes and fixes: - method-est is missing on the enrollment profile. Fix: `configure terminal crypto pki profile enrollment EST-PROFILE method-est end` - enrollment url is missing the trailing slash or uses an IP instead of the FQDN that matches the server TLS cert. - Router does not trust the server TLS cert because EST-ROOT is not yet authenticated with the Root PEM. Authenticate EST-ROOT first (see Chapter 6).

Enroll keeps failing with SIGNER_NOT_AUTHORIZED

Symptom: - est.log shows Request was rejected:

I18N_OPENXPKI_UI_ENROLLMENT_ERROR_SIGNER_NOT_AUTHORIZED. - workflows.log either shows Trusted Signer chain validation FAILED or no new workflow at all.

This has two distinct root causes. Work through them in order.

Cause A - Apache is asking for a client cert

IOS XE, when the server presents a CertificateRequest, answers with its self-signed device certificate. OpenXPKI treats that as an on-behalf signer and rejects because the self-signed cert does not chain to a trusted signer.

Fix: 1. Set `SSLVerifyClient none` on the global vhost in `apache2-openxpki-site.conf`. 2. Re-enable client-cert verification only for `/simplereenroll: apache <LocationMatch "/^/\.well-known/est/(.+)/?simplereenroll"> SSLVerifyClient optional_no_ca SSLVerifyDepth 3 SSLOptions +StdEnvVars +ExportCertData +OptRenegotiate </LocationMatch>` 3. In `est/default.yaml`, set `eligible.onbehalf.value: 0`. 4. `docker restart OpenXPKI_WebUI`.

Cause B - Stale transaction ID from a previous failed workflow

OpenXPKI derives the EST “transaction ID” from the CSR (specifically the `tid` field tied to the key). When a workflow with that TID has already failed, subsequent requests with the same key and subject are picked up from the database and return the same failure - even if you have since fixed the underlying policy.

Fix: regenerate the RSA key pair on the router so a new TID is produced:

```
configure terminal
crypto key zeroize rsa EST-CLIENT-KEY
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
end
```

Then re-run `crypto pki enroll EST-CLIENT`.

Alternative: from the CA host, find the stuck workflow and fail it:

```
docker exec OpenXPKI_Server oxi api search_workflow_instances \
  --realm democa --limit 5
docker exec OpenXPKI_Server oxi workflow execute \
  --realm democa --id <wf_id> --action workflow_force_failed
```

IOS XE “Include an IP address” prompt freezes automation

Symptom: an expect script or an SSH pipeline hangs forever when driving `crypto pki enroll`.

Cause: IOS XE emits a sequence of prompts that differ slightly between trains. The 17.18 prompts are:

1. % Include the router serial number in the subject name? [yes/no]:
2. % Include an IP address in the subject name? [no]:
3. Using EST to request certificate from CA? [yes/no]:

Note that prompt (2) ends with [no]:, not [yes/no]:. Automation patterns that look for [yes/no]: alone will miss this prompt and the session deadlocks.

Fix: match on the open bracket and a trailing colon, e.g.

```
expect -re "IP address.*:"
```

See `scripts/enroll.exp` for a tested reference.

Clock skew errors

Symptom: - `openssl s_client` reports `Verify return code: 9 (certificate is not yet valid)`. - EST enrollment fails with a generic TLS error.

Fix: align time on both hosts.

On the CA host:

```
timedatectl  
sudo systemctl restart systemd-timesyncd
```

On the router:

```
configure terminal  
 ntp server pool.ntp.org  
 clock timezone UTC 0  
end
```

If NTP is unavailable, set manually:

```
clock set 20:53:00 Apr 22 2026
```

Hostname mismatch during TLS handshake

Symptom: - Router debug shows `TLS handshake error` or `SSL verify failed`. - OpenSSL client shows `Verify return code: 62 (Hostname mismatch)`.

Causes: - The CA TLS cert CN is `svs-rtp-ca2` and the router's enrollment url uses `svs-rtp-ca2.cisco.com`.
- The CA TLS cert has no SAN at all. - You are using the IP address in the URL but the SAN only has a DNS entry (or vice versa).

Fix: re-issue the server cert with both a matching CN and a SAN that covers every hostname and IP the router might use. See Chapter 4.

Reenrollment fails with TLS handshake failure

Symptom: /simpleenroll works, but /simplereenroll fails during the TLS renegotiation step.

Causes: - The TLS session negotiated TLS 1.3, which does not support classic renegotiation. Apache cannot upgrade the connection to ask for a client cert. - The router presents a certificate that does not chain to what Apache trusts (check SSLCACertificatePath and that c_rehash ran).

Fixes: - Limit the <LocationMatch> for /simplereenroll to TLS 1.2:

```
apache <LocationMatch "^/\..well-known/est/(.+)/?simplereenroll">
    SSLProtocol -all +TLSv1.2
SSLVerifyClient optional_no_ca
SSLOptions +StdEnvVars +ExportCertData
+OptRenegotiate </LocationMatch>
```

 - Ensure the Issuing CA's certificate lives in openxpki-config/tls/chain/ so Apache will accept the router's cert as valid chain input, and run `openssl rehash openxpki-config/tls/chain/`.

Reenrollment returns the same certificate

Symptom: - `crypto pki enroll EST-CLIENT` completes without error, but show `crypto pki certificates EST-CLIENT` still shows the same serial number, start date and expiry as before. - `workflows.log` on the CA shows no new persisted `csr` or `start cert issue` entry for that subject.

Cause: OpenXPki deduplicates re-enrollment requests on `SubjectPublicKeyInfo`. When the CSR's public key matches the public key of the certificate the device is currently presenting in its mutual-TLS client auth, the server returns the existing certificate rather than issuing a new one. This protects against replayed CSRs but looks like a silent no-op the first time you hit it.

Fix: force the router to present a fresh CSR by regenerating the RSA key pair first.

Manual one-off:

```
configure terminal
crypto key zeroize rsa EST-CLIENT-KEY
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
crypto pki enroll EST-CLIENT
end
```

Timer-driven: add `regenerate` to the trustpoint's `auto-enroll` line:

```
crypto pki trustpoint EST-CLIENT
  auto-enroll 80 regenerate
```

With `regenerate` in place, each automatic renewal cycle produces a new key, a new CSR and a brand-new certificate with a different serial.

Reenrollment rejected as outside the renewal window

Symptom: - `crypto pki enroll EST-CLIENT` returns %Error in connection to Certificate Authority: Attempt to request a certificate failed: status = FAIL even though the existing certificate is still valid. - `est.log` or `workflows.log` mentions a rejected renewal or an “out of renewal window” message.

Cause: the `renewal_period` in `realm/democa/est/default.yaml` sets how far from the certificate’s notAfter OpenXPki is willing to accept a renewal. The default “000060” means the renewal window only opens 60 days before expiry, which makes “renew right after issuance” tests fail.

Fix: widen the window for the lab. For a 1-year cert, a full year works:

```
# DDDhh - 010000 = one year wide window.  
renewal_period: "010000"
```

Reload the server and client so the change takes effect:

```
docker exec OpenXPki_Server openxpkictl reload server  
docker exec OpenXPki_Client openxpkictl reload client
```

In production you typically want `renewal_period` to match or slightly exceed the portion of the certificate lifetime during which you expect the client to renew (`100% - auto_enroll_threshold`).

Auto-regenerate fails with “Querying key pair failed”

Symptom: - `show` logging around the renewal time shows: %PKI-6-CERT_RENEW_AUTO: Auto renewing router certificate of trustpoint EST-CLIENT %CRYPTO_ENGINE-5-KEY_ADDITION: A key named EST-CLIENT-KEY# has been generated %CRYPTO_ENGINE-5-KEY_ADDITION: A key named EST-CLIENT-KEY# has been generated %PKI-3-QUERY_KEY: Querying key pair failed. - The cert on the router never rolls over to a new serial; eventually it expires.

Cause: the `rsa` directive on the trustpoint uses two bit-count arguments, which puts the trustpoint into `usage-keys` mode (separate signing and encryption keys), while the actual key on disk is a single general-purpose key. When `auto-enroll regenerate` tries to build the replacement key pair it cannot match the trustpoint expectation and the renewal aborts.

Fix: use a single bit count in `rsa` for a general-purpose key.

```
crypto pki trustpoint EST-CLIENT  
no rsa  
rsa EST-CLIENT-KEY 3072
```

Use `rsa LABEL 3072 3072` only when the trustpoint is deliberately paired with a `crypto key generate rsa usage-keys` (two-key) pair.

Can't change RSA key type here

Symptom: - After editing `rsakeypair EST-CLIENT-KEY 3072` on a trustpoint that previously had `rsakeypair EST-CLIENT-KEY 2048`, the CLI rejects the change with: `% Can't change RSA key type here. % Use "crypto key generate" to change to different key type.`

Cause: IOS XE treats a change of bit size as a key-type change and refuses it inline on the trustpoint.

Fix: drop the current directive and re-add it, ideally after zeroizing and regenerating the key at the target size.

```
configure terminal
crypto pki trustpoint EST-CLIENT
no rsakeypair
rsakeypair EST-CLIENT-KEY 3072
exit
crypto key zeroize rsa EST-CLIENT-KEY
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
end
```

Then re-authenticate and re-enroll as usual.

OpenXPki refuses to parse the CSR

Symptom: - Log line: `ERR Unable to parse PKCS10: Invalid format for request: SEQUENCE not present.`

Cause: the CSR was posted as PEM instead of base64-encoded DER. The EST spec requires base64 of the DER encoding, without headers.

Fix: convert correctly before posting:

```
openssl req -in my.csr -outform DER | base64 > my.b64
```

And post the `.b64` file with `Content-Transfer-Encoding: base64`.

Useful log excerpts to grep

Log	What to look for
<code>est.log</code>	Authenticated client DN, Request was rejected, Request body is empty, the <code>rid=</code> and <code>tid=</code> for correlation.
<code>workflows.log</code>	Rendering subject, Eligibility check, Approval points, persisted csr, start cert issue, Certificate ... issued.
<code>catchall.log</code>	Trusted Signer chain validation FAILED, and any generic application errors.
Apache logs	Live only in docker logs <code>OpenXPki_WebUI</code> unless you mount <code>/var/log/apache2/</code> to a named volume.

Quickly reset the lab

If you need to wipe state and start over:

```
cd ~/est-lab/openxpki-docker
docker compose down -v          # removes the database volume
rm -rf ~/est-lab/openxpki-config/tls/ententity/openxpki.crt
rm -rf ~/est-lab/openxpki-config/tls/private/openxpki.pem
docker compose up -d
# Re-run sampleconfig.sh and the TLS-cert steps
```

`docker compose down -v` is destructive - it drops the MariaDB volume and all issued certificates. Use intentionally.

Chapter 9. Day-Two Operations

A short reference for keeping the environment healthy once the initial build is complete.

Reload vs restart

Change	Command
realm/democa/**/*.yaml	<code>docker exec OpenXPKI_Server openxpkictl reload server</code> and <code>docker exec OpenXPKI_Client openxpkictl reload client</code>
Workflow profile or tls_dual profile	<code>docker compose restart server client</code>
Apache vhost	<code>docker restart OpenXPKI_WebUI</code>
TLS server certificate file on disk	<code>docker restart OpenXPKI_WebUI</code>
system/cli.yaml or system/crypto.yaml	Full stack restart: <code>docker compose restart</code>

Log locations

File	Purpose
<code>/var/log/openxpki-server/openxpki.log</code>	Application log for the server daemon
<code>/var/log/openxpki-server/workflows.log</code>	Per-workflow high-level events (the most useful file)
<code>/var/log/openxpki-server/catchall.log</code>	Cross-cutting application events
<code>/var/log/openxpki-server/audit.log</code>	Security-relevant events (CA key use, CRL issue, etc.)
<code>/var/log/openxpki-client/est.log</code>	Per-request EST client log (rid, tid, subject, rejections)
<code>/var/log/openxpki-client/scep.log</code> , <code>rpc.log</code> , <code>webui.log</code>	Protocol-specific logs
<code>docker logs OpenXPKI_WebUI</code>	Apache stdout/stderr (proxy errors, TLS alerts)

Backup

For the lab, the only stateful pieces are:

1. The MariaDB volume (openxpkiadb).
2. The host-side openxpki-config/ tree (seed config plus your edits).
3. The openxpki-config/tls/ directory (TLS server key + cert).

A minimal offline backup:

```
# Pause writes while backing up
docker compose -f ~/est-lab/openxpki-docker/docker-compose.yml stop server client web

# DB dump
docker exec OpenXPKI_Database \
    mariadb-dump --all-databases -u root > ~/est-lab-db-$(date +%F).sql

# Config tarball
tar czf ~/est-lab-config-$(date +%F).tgz -C ~/est-lab openxpki-config

docker compose -f ~/est-lab/openxpki-docker/docker-compose.yml start server client web
```

Store these somewhere outside the CA host, encrypted at rest.

Certificate housekeeping

List active end-entity certificates in the realm:

```
docker exec OpenXPKI_Server oxi api search_cert \
    --realm democa --status ISSUED --entity_only 1 --limit 100
```

Revoke a certificate (for example after replacing a device):

```
docker exec OpenXPKI_Server oxi api revoke_cert \
    --realm democa --identifier <cert_identifier> \
    --reason superseded
```

Publish an updated CRL:

```
docker exec OpenXPKI_Server oxi workflow create \
    --realm democa --type crl_issuance
```

Renewal on the router

With `auto-enroll 80 regenerate` configured on the EST-CLIENT trustpoint, the router will:

1. At 80% of the certificate's lifetime, spawn a re-enrollment workflow.
2. Generate a new key pair (`regenerate`) to avoid long-term key reuse.
3. Call `/simplereenroll` over a TLS session authenticated with the current device certificate.
4. Install the new certificate, replacing the previous one.

Two prerequisites must also be in place for `/simplereenroll` to succeed:

- The enrollment profile has to expose the device cert to the TLS stack:

```
crypto pki profile enrollment EST-PROFILE
    enrollment credential EST-CLIENT
```

- The Apache vhost in front of OpenXPki has to allow classic TLS 1.2 client-auth renegotiation for the /simplereenroll location. TLS 1.3 post-handshake client auth is not supported by the IOS XE EST client; force TLS 1.2 on that path (see examples/openxpki/apache2-openxpki-site.conf).

You can force an immediate renewal with:

```
configure terminal
crypto pki enroll EST-CLIENT
end
```

Because the router already has an active certificate, IOS XE uses the /simplereenroll path automatically. A subtle gotcha: OpenXPki returns the existing certificate rather than issuing a new one when the CSR carries the same SubjectPublicKeyInfo as the cert the device just presented. The serial number therefore does not change unless the router generates a new key first. For an on-demand test that produces a brand-new cert:

```
configure terminal
crypto key zeroize rsa EST-CLIENT-KEY
crypto key generate rsa general-keys label EST-CLIENT-KEY modulus 3072
crypto pki enroll EST-CLIENT
end
```

For timer-driven renewals this is exactly what the regenerate keyword on auto-enroll does for you - which is why every production trustpoint should carry auto-enroll N regenerate rather than a bare auto-enroll N. Without regenerate, the router keeps re-submitting the same public key, OpenXPki deduplicates on SubjectPublicKeyInfo, and the cert on disk never actually rolls.

Renewal is also bounded by the renewal_period setting in realm/democa/est/default.yaml. OpenXPki rejects re-enrollment attempts that fall outside that window measured backwards from notAfter. A lab that renews aggressively or uses very short validities should widen the window, for example renewal_period: "010000" (1 year) so every request inside the certificate's lifetime is accepted.

Updating OpenXPki

The Docker image is pinned in docker-compose.yml. To update:

```
cd ~/est-lab/openxpki-docker
git pull
# review diff to docker-compose.yml and openxpki-config/
docker compose pull
docker compose up -d
```

Before cutting over in production, test a pinned-tag upgrade on a replica.

Credential rotation

HTTP Basic bootstrap passwords in openxpki-config/htpasswd/est.passwd should rotate on a schedule. The recommended cadence and the two-hash rollover pattern are documented in Chapter 5, with the helper script at scripts/rotate-est-bootstrap-cred.sh.

A quick summary of when to rotate:

Event	Action
Routine	Rotate every 90 days
Staff change (PKI operator turnover)	Rotate all entries
Suspected leak	Rotate the affected entry immediately
Device retirement	Delete the line
One-shot bootstrap hardening	Delete the line once the device has a cert

The Issuing CA signing key (software `svault` in this lab) should be re-sealed with a new passphrase on a longer cadence, typically annually or on any change in personnel with access to it. See OpenXPKI's [datavault documentation](#) for the reseal workflow.

Hardening review checklist

The configuration walked through in chapters 3 and 5 already implements the majority of practical hardening. Before promoting the stack out of lab use, walk through what remains:

- ☒ CRL Distribution Point URL embedded in every issued certificate (chapter 3 section 8).
- ☒ `authorized_signer` pinned to profile: `tls_dual` + subject regex (chapter 5 section 1).
- ☒ `tls_dual` profile enforces an FQDN allow-list on CN and SAN (chapter 5 section 2).
- ☒ SKI/AKI extensions explicit in `tls_dual` (chapter 5 section 2).
- ☒ Full set of HTTP security headers on the WebUI vhost (chapter 5 section 3c).
- ☒ Per-device HTTP Basic credentials with a documented rotation cadence and rollover script (chapter 5 section 3b).
- ☒ HTTP Basic authentication required on `/simpleenroll` (chapter 5 section 3b).
- ☒ mTLS with the device's current certificate required on `/simplereenroll` (chapter 5 section 3d).
- ☒ `auto-enroll N regenerate` on the router trustpoint (chapter 6).

Still open for a genuine production cutover:

- ☐ Replace `DUMMY` CA hierarchy with a new Root/Issuing CA built through the production bootstrap workflow.
- ☐ Move the CA signing key from the software `svault` into an HSM/KMS via OpenXPKI's PKCS#11 token support.
- ☐ Enable `revocation-check crl` on every trustpoint on every router (already true for `EST-CLIENT` in this lab; extend it to `EST-ROOT`).
- ☐ Publish the CRL externally so every client can reach the URL that was embedded into its certificate. The CA host exposes port 8080 for this in the lab; harden that with a reverse proxy or CDN in production.
- ☐ Put the EST endpoint behind a WAF or rate-limiter (`mod_evasive`, `fail2ban` on the `est_access.log` 401s).

- ☐ Ship server, client, audit, and Apache logs to a central SIEM.
- ☐ Authenticate NTP (`ntp authentication-key / ntp server ... key ...`) on every router so an on-path attacker cannot roll the clock to extend a revoked cert or suppress an expiration.